

环境安装

安装 python

```
1 yum -y install epel-release
2 yum install -y python.x86_64
3 yum install -y python-pip
```

查看版本

```
1 python -V
2 pip -V
```

升级pip

```
1 pip install --upgrade pip
2 pip install --upgrade setuptools
```

安装Miniconda

```
1 mkdir /opt/modules/miniconda
2 cd /opt/modules/miniconda
3 wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
4 sh Miniconda3-latest-Linux-x86_64.sh
```

使用miniconda

```
1 source ~/.bashrc
2 conda config --set auto_activate_base false
3 conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/main
4 conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/free
5 conda config --set show_channel_urls yes
```

创建环境

```
1 conda create --name python3 python=3.7.4
```

切换环境

```
1 conda activate python3
```

退出环境

```
1 conda deactivate
```

虚拟环境

创建虚拟环境

```
1 python -m venv envname
```

激活虚拟环境

```
1 cd envname/Scripts
2 ./activate
```

VScode设置虚拟环境

```
1 Ctrl+Shift+P --> Python:Select Interpreter --> 选择 python.exe 文件位置
```

anaconda

源管理(4.5.1)

```
1 conda config --show channels
2 conda config --add channels "https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgsg/free/"
3 conda config --add channels "https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgsg/main/"
4 conda config --add channels "https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-
  forge/"
5
6
7 conda config --add channels
  https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/pytorch/
8 conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/menpo/
9 conda config --add channels
  https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/bioconda/
10 conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/msys2/
11 conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-
  forge/
12 conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgsg/main/
13 conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgsg/free/
14 conda config --set show_channel_urls yes
15 conda config --set ssl_verify false
16
17
18 conda config --remove channels conda-forge
19 conda config --add channels conda-forge
20 conda config --set channel_priority strict
```

添加

```
1 conda config --add channels "https://repo.continuum.io/pkgsg/main/"
```

删除

```
1 conda config --remove channel "https://repo.continuum.io/pkgsg/main/"
```

```
conda config --set show_channel_urls yes
```

搜索

模糊搜索

```
1 conda search <keyword>
2 eg conda search python
```

精确搜索

```
1 conda search --full-name <package_name>
```

环境管理

创建环境

```
1 conda create -n <env> <pkg=3.7.0> ...
2 eg :conda create -n python3.7.0 python=3.7.0
```

查看环境

```
1 conda env list
2 conda info --envs
3 conda info -e
```

使用环境

```
1 conda activate <envname>
2 eg:conda activate python3.7.0
```

退出环境

```
1 conda deactivate
```

复制环境

```
1 conda create --name <newenvname> --clone <orienvname>
```

删除环境

```
1 conda remove -n <envname> --all
```

包管理

在当前环境中安装包

```
1 conda install <package_name>
```

在当前环境中安装包(指定版本)

```
1 conda install <package_name>=version
```

在指定环境中安装包

```
1 conda install --name <envname> <package_name>=version
```

使用pip安装包

```
1 pip install <package_name>
```

包管理

安装

```
1 #特定版本号
2 pip install "package==version"
3 #范围安装
4 pip install "package>2.5,<3.0"
5 #不指定版本号，安装最新版本
6 pip install package
7 #批量安装
8 pip install -r requirement.txt
9 #导出包配置
10 pip freeze > requirement.txt
11 #加速安装
12 pip install packagename -i http://mirrors.aliyun.com/pypi/simple/ --trusted-host
    mirrors.aliyun.com
13
14 #其他加速镜像
15 pip install numpy pandas -i https://pypi.tuna.tsinghua.edu.cn/simple
16 pip install numpy pandas -i https://mirrors.ustc.edu.cn/pypi/web/simple
```

安装加速全局配置

在 %APPDATA%/pip/pip.ini 文件中添加

```
1 [global]
2 index-url=http://mirrors.aliyun.com/pypi/simple/
3 [install]
4 trusted-host=mirrors.aliyun.com
```

命令行设置

```
1 pip config set global.index-url https://pypi.doubanio.com/simple
```

uv

安装

```
1 pip install uv
2
3 #设置镜像
4 set UV_DEFAULT_INDEX "https://mirrors.aliyun.com/pypi/simple/"
5 #或手动设置环境变量
```

使用


```
1 cd 项目目录
2 uv init --no-package
3 uv venv #创建虚拟环境目录
4 uv add package #安装包
5 uv tree #安装包查看
6 uv run main.py #执行项目
7 #分发：打包除.venv目录的所有文件
8 #分发后安装
9 uv sync
10 #删除包
11 uv remove 包名
12 #安装指定的版本
13 uv python install cpython3.12
14 uv python pin 3.12 #指定python版本号
15 uv python list #列出已安装/未安装的python版本
16 uv self update #更新uv自身
17 uv uv self unstall #卸载
18 uv tool install ruff #安装uv配套工具ruff
```

基本语法

1. 在Python中严格区分大小写
2. Python中的每一行就是一条语句，每条语句以换行结束
3. Python中每一行语句不要过长（规范中建议每行不要超过80个字符）
4 "rulers": [80],
4. 一条语句可以分多行编写，多行编写时语句后边以\结尾
5. Python是缩进严格的语言，所以在Python中不要随便写缩进
6. 在Python中使用#来表示注释，#后的内容都属于注释，注释的内容将会被解释器所忽略
8 我们可以通过注释来对程序进行解释说明，一定要养成良好的编写注释的习惯
9 注释要求简单明了，一般习惯上#后边会跟着一个空格

变量与标识符

```
1 # Python中使用变量，不需要声明，直接为变量赋值即可
2 a = 100
3
4 # 不能使用没有进行过赋值的变量
5 # 如果使用没有赋值过的变量，会报错 NameError: name 'b' is not defined
6 # print(b)
7
8 # Python是一个动态类型的语言，可以为变量赋任意类型的值，也可以任意修改变量的值
9 a = 'hello'
10
11 # print(a)
12
13 # 标识符
14 # 在Python中所有可以自主命名的内容都属于标识符
15 # 比如：变量名、函数名、类名
16 # 标识符必须遵循标识符的规范
17 # 1.标识符中可以含有字母、数字、_，但是不能使用数字开头
18 #     例子：a_1 _a1 _1a
19 # 2.标识符不能是Python中的关键字和保留字
20 #     也不建议使用Python中的函数名作为标识符,因为这样会导致函数被覆盖
21 # 3.命名规范：
22 #     在Python中注意遵循两种命名规范：
23 #         下划线命名法
24 #             所有字母小写，单词之间使用_分割
25 #             max_length min_length hello_world xxx_yyy_zzz
26 #         帕斯卡命名法（大驼峰命名法）
27 #             首字母大写，每个单词开头字母大写，其余字母小写
28 #             MaxLength MinLength HelloWorld XxxYyyZzz
29 #
30 # 如果使用不符合标准的标识符，将会报错 SyntaxError: invalid syntax
31 # 练习：尝试自己定义几个变量（复杂一些，尝试不同的命名法），然后打印一些变量
32 #     通过搜索引擎搜索还有哪些其他的命名规范
33 _b123 = 20
34 # print(_b123)
35
36 # print = 123
37 # print(print)
```

换行与注释

```
1 print('he\  
2 aaa\  
3 aaa')  
4  
5 # 这是一个打印语句，请你看见了不要慌张  
6 # 这是一个注释  
7 # 注释会被解释器所忽略  
8 # print(123+456) 这行代码被注释了，将不会执行  
9 print('abc') # 这是一个单行注释
```

数据类型

数值

[illegible]

分隔

```
1 s = "Hello,World"
2 result = s.split(",")
3 print(result) # 输出: ['Hello', 'World']
```

替换

```
1 s = "Hello,World"
2 result = s.replace("World", "Python")
3 print(result) # 输出: Hello,Python
```

查找

```
1 s = "Hello,World"
2 result = s.find("World")
3 print(result) # 输出: 6
```

计数

```
1 s = "Hello,World,World"
2 result = s.count("World")
3 print(result) # 输出: 2
```

大小写转换

```
1 s = "Hello,World"
2 result_upper = s.upper()
3 result_lower = s.lower()
4 print(result_upper) # 输出: HELLO,WORLD
5 print(result_lower) # 输出: hello,world
```

首字母大写

```
1 s = "hello,world"
2 result = s.capitalize()
3 print(result) # 输出: Hello,world
```

字符串居中对齐

```
1 s = "Hello,World"
2 result = s.center(15, "-")
3 print(result) # 输出: --Hello,World--
```

字符串左对齐

```
1 s = "Hello,World"
2 result = s.ljust(20, "-")
3 print(result) # 输出: Hello,World-----
```

字符串右对齐

```
1 s = "Hello,World"
2 result = s.rjust(20, "-")
3 print(result) # 输出: -----Hello,World
```

去除空格

```
1 s = " Hello,World "
2 result = s.strip()
3 print(result) # 输出: Hello,World
```

判断是否以指定子串开头

```
1 s = "Hello,World"
2 result = s.startswith("Hello")
3 print(result) # 输出: True
```

判断是否以指定子串结尾

```
1 s = "Hello,World"
2 result = s.endswith("World")
3 print(result) # 输出: True
```

判断是否全为数字

```
1 s = "123"
2 result = s.isdigit()
3 print(result) # 输出: True
```

判断是否全为字母

```
1 s = "abc"
2 result = s.isalpha()
3 print(result) # 输出: True
```

判断是否全为空白字符

```
1 s = " "
```

```
2 result = s.isspace()
3 print(result) # 输出: True
```

判断是否为标题格式

```
1 s = "Hello World"
2 result = s.istitle()
3 print(result) # 输出: True
```

判断是否为小写字母

```
1 s = "hello"
2 result = s.islower()
3 print(result) # 输出: True
```

判断是否为大写字母

```
1 s = "HELLO"
2 result = s.isupper()
3 print(result) # 输出: True
```

判断是否为标识符

```
1 s = "hello"
2 result = s.isidentifier()
3 print(result) # 输出: True
```

判断是否为数字类型

```
1 s = "123"
2 result = s.isnumeric()
3 print(result) # 输出: True
```


拼接

```
1 str1 = "Hello"
2 str2 = "World"
3 result = str1 + str2
4 print(result) # 输出: HelloWorld
```

字符串复制

```
1 str1 = "Hello"
2 result = str1 * 3
3 print(result) # 输出: HelloHelloHello
```

字符串长度

```
1 str1 = "Hello"
2 length = len(str1)
3 print(length) # 输出: 5
```

字符串格式化

```
1 name = "Tom"
2 age = 18
3 result = "My name is {} and I am {} years old.".format(name, age)
4 print(result) # 输出: My name is Tom and I am 18 years old.
5
6 # 占位符
7 print('欢迎 %s 光临! '%name)
8 # 格式化字符串
9 print(f'欢迎 {name} 光临! ')
```

字符串切片

```
1 str1 = "Hello,World"
2 result = str1[0:5]
3 print(result) # 输出: Hello
```

字符串连接

```
1 str1 = "Hello"
2 str2 = "World"
3 result = "-".join([str1, str2])
4 print(result) # 输出: Hello-World
```

获取字符串中的数字

```
1 import re
2
3 def get_numbers_from_string(s):
4     return re.findall(r'\d+', s)
5
6 s = "abc123def456"
7 numbers = get_numbers_from_string(s)
8 print(numbers) # 输出: ['123', '456']
```

获取字符串中的字母

```
1 import re
2
3 def get_letters_from_string(s):
4     return re.findall(r'[a-zA-Z]+', s)
5
6 s = "abc123def456"
7 letters = get_letters_from_string(s)
8 print(letters) # 输出: ['abc', 'def']
```

反转字符串

```
1 s = "Hello, World!"
2 reversed_s = s[::-1]
3 print(reversed_s) # 输出: !dlrow ,olleH
```

其他操作

```
1 # 字符串(str)
2 # 字符串用来表示一段文本信息，字符串是程序中使用的最多的数据类型
3 # 在Python中字符串需要使用引号引起来
4
5 s = 'hello'
6 # s = abc # 字符串必须使用引号引起来，不使用不是字符串
7 # 引号可以是双引号，也可以是单引号，但是注意不要混着用
8 s = 'hello'
9 s = "hello"
10 # s = 'hello" 引号不能混合使用 SyntaxError: EOL while scanning string literal
11
12 # 相同的引号之间不能嵌套
13 # s = "子曰:"学而时习之，乐呵乐呵！""
14 s = '子曰:"学而时习之，乐呵乐呵！"'
15
16 # 长字符串
17 # 单引号和双引号不能跨行使用
18 s = '锄禾日当午，\
19 汗滴禾下土，\
20 谁知盘中餐，\
21 粒粒皆辛苦'
22
23 # 使用三重引号来表示一个长字符串 ''' '''
24 # 三重引号可以换行，并且会保留字符串中的格式
25
26 s = '''锄禾日当午，
27 汗滴禾下土，
28 谁知盘中餐，
29 粒粒皆辛苦'''
30
31 # 转义字符
32 # 可以使用 \ 作为转义字符，通过转义字符，可以在字符串中使用一些特殊的内容
33 # 例子：
34 # \ ' 表示'
35 # \" 表示"
36 # \t 表示制表符
37 # \n 表示换行符
38 # \\ 表示反斜杠
```

```
39 # \uxxxx 表示Unicode编码
40 s = "子曰:\\"学而时习之，\\\n乐呵乐呵!\\"
41
42 s = '\u2250'
43 print(s)
```

bool 与 None

```
1 # 布尔值（bool）
2 # 布尔值主要用来做逻辑判断
3 # 布尔值一共有两个 True 和 False
4 # True表示真 False表示假
5 a = True
6 a = False
7 # print('a =',a)
8
9 # 布尔值实际上也属于整型，True就相当于1，False就相当于0
10 # print(1 + False)
11
12 # None（空值）
13 # None专门用来表示不存在
14 b = None
15 print(b)
```

类型检查

```
1 # 通过类型检查，可以检查只能值（变量）的类型
2
3 a = 123 # 数值
4 b = '123' # 字符串
5
6 # print('a =',a)
7 # print('b =',b)、
8
9 # type()用来检查值的类型
10 # 该函数会将检查的结果作为返回值返回，可以通过变量来接收函数的返回值
11 c = type('123')
12 c = type(a)
13 # print(type(b))
14 print(type(1)) # <class 'int'>
15 print(type(1.5)) # <class 'float'>
16 print(type(True)) # <class 'bool'>
17 print(type('hello')) # <class 'str'>
18 print(type(None)) # <class 'NoneType'>
```

类型转换

```
1 # 类型转换四个函数 int() float() str() bool()
2 # int() 可以用来将其他的对象转换为整型
3 # 规则:
4 #   布尔值: True -> 1   False -> 0
5 #   浮点数: 直接取整, 省略小数点后的内容
6 #   字符串: 合法的整数字符串, 直接转换为对应的数字
7 #           如果不是一个合法的整数字符串, 则报错 ValueError: invalid literal for int()
            with base 10: '11.5'
8 #   对于其他不可转换为整型的对象, 直接抛出异常 ValueError
9 # float() 和 int()基本一致, 不同的是它会将对象转换为浮点数
10 # str() 可以将对象转换为字符串
11 #   True -> 'True'
12 #   False -> 'False'
13 #   123 -> '123'
14 #   ...
15 # bool() 可以将对象转换为布尔值, 任何对象都可以转换为布尔值
16 #   规则: 对于所有表示空性的对象都会转换为False, 其余的转换为True
17 #       哪些表示的空性: 0 、 None 、 '' ...
18
19 a = True
20
21 # 调用int()来将a转换为整型
22 # int()函数不会对原来的变量产生影响, 他是对象转换为指定的类型并将其作为返回值返回
23 # 如果希望修改原来的变量, 则需要对变量进行重新赋值
24 a = int(a) //1
25
26 a = False
27 a = int(a) //0
28
29 a = '123'
30 a = int(a)
31
32 a = 11.6
33 a = int(a) //11
34
35 a = '11.5'
36 # a = int(a) //error
37
```

```
38 a = None
39 # a = int(a) //error
40
41 a = 1
42 a = float(a) //1.0
43
44 a = False
45 a = float(a) //0.0
46
47 a = 123
48 a = str(a)
49
50 a = None
51 a = bool(a) //False
52
53 print('a =',a)
54 print('a的类型是',type(a))
55 # b = 456
56 # print('hello'+str(b))
57
58 a=''
59 a=bool(a) //False
60
61 a=' '
62 a=bool(a) //True
```

运算符

算术运算符


```
1 # 算术运算符
2 # + 加法运算符（如果是两个字符串之间进行加法运算，则会进行拼串操作）
3 # - 减法运算符
4 # * 乘法运算符（如果将字符串和数字相乘，则会对字符串进行复制操作，将字符串重复指定次数）
5 # / 除法运算符，运算时结果总会返回一个浮点类型
6 # // 整除，只会保留计算后的整数位，总会返回一个整型
7 # ** 幂运算，求一个值的几次幂
8 # % 取模，求两个数相除的余数
9
10 a = 10 + 5 # 计算
11 a = 'hello' + ' ' + 'world' # 拼串
12
13 a = 10 - 5 # 计算
14 a = 5 - True
15 a = a - 2 # 用变量a的值减去2，然后再赋值给a
16 # a = 'hello' - 'h' TypeError
17
18 a = 5 * 5
19
20 a = 10 / 5
21 a = 5 / 2
22 # a = 5 / 0 ZeroDivisionError: division by zero
23 a = 10 / 3
24 a = 10 // 3
25 a = 5 // 2
26
27 a = 2 ** 2
28 a = 10 ** 5
29 a = 16 ** 0.5 # 求16的平方根
30
31 a = 10 % 5 # 0
32 a = 10 % 4 # 2
33 a = 10 % 3 # 1
34 a = 10 % 2 # 0
35
36 print("a =", a)
```

赋值运算符

```
1 # 赋值运算符
2 # = 可以将等号右侧的值赋值给等号左侧的变量
3 # += a += 5 相当于 a = a + 5
4 # -= a -= 5 相当于 a = a - 5
5 # *= a *= 5 相当于 a = a * 5
6 # **= a **= 5 相当于 a = a ** 5
7 # /= a /= 5 相当于 a = a / 5
8 # //= a //= 5 相当于 a = a // 5
9 # %= a %= 5 相当于 a = a % 5
10 a = 10
11 # a = a + 5
12 # a += 5
13 a -= 5
14 a *= 5
15 a **= 2
16 a /= 25
17 a = 25.0 # 在对浮点数做算术运算时，结果也会返回一个浮点数
18 a //= 5
19 a = 5
20 a %= 4
21
22 print('a =', a)
23
```

关系运算符

```
1 # 关系运算符
2 # 关系运算符用来比较两个值之间的关系，总会返回一个布尔值
3 # 如果关系成立，返回True，否则返回False
4 # > 比较左侧值是否大于右侧值
5 # >= 比较左侧的值是否大于或等于右侧的值
6 # < 比较左侧值是否小于右侧值
7 # <= 比较左侧的值是否小于或等于右侧的值
8 # == 比较两个对象的值是否相等
9 # != 比较两个对象的值是否不相等
10 # 相等和不等比较的是对象的值，而不是id
11 # is 比较两个对象是否是同一个对象，比较的是对象的id
12 # is not 比较两个对象是否不是同一个对象，比较的是对象的id
13 result = 10 > 20 # False
14 result = 30 > 20 # True
15 result = 30 < 20 # False
16 result = 10 >= 10 # True
17
18 result = 2 > True # True
19 # result = 2 > '1' TypeError: '>' not supported between instances of 'int' and 'str'
20
21 # 0032 > 0031
22 result = '2' > '1' # True
23 result = '2' > '11' # True
24
25 # 在Python中可以对两个字符串进行大于（等于）或小于（等于）的运算，
26 # 当对字符串进行比较时，实际上比较的是字符串的Unicode编码
27 # 比较两个字符串的Unicode编码时，是逐位比较的
28 # 利用该特性可以对字符串按照字母顺序进行排序，但是对于中文来说意义不是特别大
29 # 注意：如果不希望比较两个字符串的Unicode编码，则需要将其转换为数字然后再比较
30 # 0061 > 0062
31 result = 'a' > 'b' # False
32 result = 'c' < 'd' # True
33 result = 'ab' > 'b' # False
34
35 # print(int('2') > int('11'))
36
37 result = 1 == 1 # True
38 result = 'hello' == 'hello' # True
```

```
39 result = 'abc' == 'bcd' # False
40 result = 'abc' != 'bcd' # True
41 result = 1 == True # True
42 result = 1 is True # False
43 result = 1 is not True # True
44 print('result =',result)
45 print(id(1),id(True))
```

逻辑运算符

```
1 # 逻辑运算符
2 # 逻辑运算符主要用来做一些逻辑判断
3 # not 逻辑非
4 #     not可以对符号右侧的值进行非运算
5 #         对于布尔值，非运算会对其进行取反操作，True变False，False变True
6 #         对于非布尔值，非运算会先将其转换为布尔值，然后再取反
7 #
8 # and 逻辑与
9 #     and可以对符号两侧的值进行与运算
10 #     只有在符号两侧的值都为True时，才会返回True，只要有一个False就返回False
11 #     与运算是找False的
12 #     Python中的与运算是短路的与，如果第一个值为False，则不再看第二个值
13 #
14 # or 逻辑或
15 #     or 可以对符号两侧的值进行或运算
16 #     或运算两个值中只要有一个True，就会返回True
17 #     或运算是找True的
18 #     Python中的或运算是短路的或，如果第一个值为True，则不再看第二个值
19 #
20 # 练习：
21 #     尝试一下对布尔值进行三种逻辑运算
22 #     尝试对非布尔值进行三种逻辑运算，并观察返回的结果
23 #
24
25 a = True
26 a = not a # 对a进行非运算
27
28 a = 1
29 a = ''
30 a = not a
31 # print('a =',a)
32
33 result = True and True # True
34 result = True and False # False
35 result = False and True # False
36 result = False and False # False
37
38 # print(result)
```

```
39
40 # True and print('你猜我出来吗? ') 第一个值是True, 会看第二个值, 所以print()会执行
41 # False and print('你猜我出来吗? ') 第一个值是False, 不会看第二个值, 所以print()不会执行
42
43
44 result = True or True # True
45 result = True or False # True
46 result = False or True # True
47 result = False or False # False
48
49 # print(result)
50 # False or print('你猜我出来吗? ') 第一个值为False, 继续看第二个, 所以打印语句执行
51 # True or print('你猜我出来吗? ') 第一个值为True, 不看第二个, 所以打印语句不执行
52
53 # 非布尔值的与或运算
54 #  当我们对非布尔值进行与或运算时, Python会将其当做布尔值运算, 最终会返回原值
55 #  与运算的规则
56 #      与运算是找False的, 如果第一个值是False, 则不看第二个值
57 #      如果第一个值是False, 则直接返回第一个值, 否则返回第二个值
58 #  或运算的规则
59 #      或运算是找True的, 如果第一个值是True, 则不看第二个值
60 #      如果第一个值是True, 则直接返回第一个值, 否则返回第二个值
61
62 # True and True
63 result = 1 and 2 # 2
64 # True and False
65 result = 1 and 0 # 0
66 # False and True
67 result = 0 and 1 # 0
68 # False and False
69 result = 0 and None # 0
70
71 # True or True
72 result = 1 or 2 # 1
73 # True or False
74 result = 1 or 0 # 1
75 # False or True
76 result = 0 or 1 # 1
77 # False or False
```

```
78 result = 0 or None # None
79
80 print(result)
81
82 a=30
83 a=not not a
84 print("a=",a) #true
85
86 a=''
87 a=not not a
88 print("a=",a) #false
89 a=' '
90 a = not not a
91 print("a=",a) #true
```

条件运算符

```
1 # 条件运算符（三元运算符）
2 # 语法： 语句1 if 条件表达式 else 语句2
3 # 执行流程：
4 #     条件运算符在执行时，会先对条件表达式进行求值判断
5 #     如果判断结果为True，则执行语句1，并返回执行结果
6 #     如果判断结果为False，则执行语句2，并返回执行结果
7 # 练习：
8 #     现在有a b c三个变量，三个变量中分别保存有三个数值，
9 #     请通过条件运算符获取三个值中的最大值
10
11 # print('你好') if False else print('Hello')
12
13 a = 30
14 b = 50
15
16 # print('a的值比较大! ') if a > b else print('b的值比较大! ')
17 # 获取a和b之间的较大值
18 max = a if a > b else b
19 print(max)
```

运算符优先级


```
1 a = 40
2 b = 50
3 c = 30
4
5 # 通过条件运算符获取三个值中的最大值
6 # max = a if a > b else b
7 # max = max if max > c else c
8
9 max = a if (a > b and a > c) else (b if b > c else c) # 不推荐这么使用
10 # max = a if (b < a > c) else (b if b > c else c)
11
12 # print(max)
13
14 # 运算符的优先级
15 # 和数学中一样，在Python运算也有优先级，比如先乘除 后加减
16 # 运算符的优先级可以根据优先级的表格来查询，
17 # 在表格中位置越靠下的运算符优先级越高，优先级越高的越优先计算
18 # 如果优先级一样则自左向右计算
19 # 关于优先级的表格，你知道有这么一个东西就够了，千万不要去记
20 # 在开发中如果遇到优先级不清楚的，则可以通过小括号来改变运算顺序
21 a = 1 + 2 * 3
22
23 # 一样 and高 or高
24 # 如果or的优先级高，或者两个运算符的优先级一样高
25 # 则需要先进行或运算，则运算结果是3
26 # 如果and的优先级高，则应该先计算与运算
27 # 则运算结果是1
28 a = 1 or 2 and 3
29
30 # print(a)
31
32 # 逻辑运算符（补充）
33 # 逻辑运算符可以连着使用
34 result = 1 < 2 < 3 # 相当于 1 < 2 and 2 < 3
35 result = 10 < 20 > 15
36
37 print(result)
```

流程控制

if

```
1 # 条件判断语句（if语句）
2 # 语法：if 条件表达式：
3 #         代码块
4 # 执行的流程：if语句在执行时，会先对条件表达式进行求值判断，
5 #     如果为True，则执行if后的语句
6 #     如果为False，则不执行
7 # 默认情况下，if语句只会控制紧随其后的那条语句，如果希望if可以控制多条语句，
8 #     则可以在if后跟着一个代码块
9 # 代码块
10 #     代码块中保存着一组代码，同一个代码块中的代码，要么都执行要么都不执行
11 #     代码块就是一种为代码分组的机制
12 #     如果要编写代码块，语句就不能紧随在:后边，而是要写在下一行
13 #     代码块以缩进开始，直到代码恢复到之前的缩进级别时结束
14 #     鲁迅说过：
15 #         世上本来没有路，走的人多了自然就有了！
16 #         xxxx
17 #     yyyy....
18 # 缩进有两种方式，一种是使用tab键，一种是使用空格（四个）
19 #     Python的官方文档中推荐我们使用空格来缩进
20 #     Python代码中使用的缩进方式必须统一
21 #     "translate_tabs_to_spaces": true,
22
23 # if False : print('你猜我出来么? ')
24
25 num = 10
26 # if num > 10 : print('num比10大! ')
27 # print('谁也管不了我')
28
29 if False :
30     print(123)
31     print(456)
32     print(789)
33     print(101112)
34 # print('hello')
35
36 num = 28
37
38 # 可以使用逻辑运算符来连接多个条件，
```

```
39 # 如果希望所有条件同时满足,则需要使用and
40 # 如果希望只要有一个条件满足即可,则需要使用or
41 # if num > 10 and num < 20 :
42 #     print('num比10大,num比20小! ')
43
44 # if 10 < num < 20 :
45 #     print('num比10大,num比20小! ')
46
47 # 在命令行让用户输入一个用户名,获取用户输入,并进行判断
48 # 如果用户输入的用户名是admin,则显示欢迎管理员光临
49 # 如果用户输入的是其他的用户名,则什么也不做
```

```
1 # 让用户在控制台中输入一个年龄
2 # age = int(input('请输入你的年龄: '))
3
4 # 如果用户的年龄大于18岁,则显示你已经成年了
5 # if age >= 18 :
6 #     print('你已经成年了~~~')
7
8 # if-else语句
9 # 语法:
10 # if 条件表达式 :
11 #     代码块
12 # else :
13 #     代码块
14 # 执行流程:
15 # if-else语句在执行时,先对if后的条件表达式进行求值判断
16 #     如果为True,则执行if后的代码块
17 #     如果为False,则执行else后的代码块
18
19 age = 7
20 if age > 17 :
21     print('你已经成年了~~')
22 else :
23     print('你还未成年~~')
```

```
1 # if-elif-else语句
2 # 语法:
3 #     if 条件表达式 :
4 #         代码块
5 #     elif 条件表达式 :
6 #         代码块
7 #     elif 条件表达式 :
8 #         代码块
9 #     elif 条件表达式 :
10 #         代码块
11 #     else :
12 #         代码块
13 #
14 # 执行流程:
15 #     if-elif-else语句在执行时, 会自上向下依次对条件表达式进行求值判断,
16 #         如果表达式的结果为True, 则执行当前代码块, 然后语句结束
17 #         如果表达式的结果为False, 则继续向下判断, 直到找到True为止
18 #         如果所有的表达式都是False, 则执行else后的代码块
19 #     if-elif-else中只会会有一个代码块会执行
20
21 age = 210
22
23 # if age > 200 :
24 #     print('活着可真没劲呢! ')
25 # elif age > 100 :
26 #     print('你也是老大不小了! ')
27 # elif age >= 60 :
28 #     print('你已经退休了! ')
29 # elif age >= 30 :
30 #     print('你已经是中年了! ')
31 # elif age >= 18 :
32 #     print('你已经成年了! ')
33 # else :
34 #     print('你还是个小孩! ')
35
36 age = 68
37
38 if age >= 18 and age < 30 :
```

```
39     print('你已经成年了! ')\n40 elif age >= 30 and age < 60 : \n41     print('你已经中年了! ')\n42 elif age >= 60 : \n43     print('你已经退休了! ')
```

while

```
1 # 循环语句
2 # 循环语句可以使指定的代码块重复指定的次数
3 # 循环语句分成两种，while循环 和 for循环
4 # while循环
5 # 语法：
6 #     while 条件表达式 :
7 #         代码块
8 #     else :
9 #         代码块
10 # 执行流程：
11 #     while语句在执行时，会先对while后的条件表达式进行求值判断，
12 #         如果判断结果为True，则执行循环体（代码块），
13 #         循环体执行完毕，继续对条件表达式进行求值判断，以此类推，
14 #         直到判断结果为False，则循环终止，如果循环有对应的else，则执行else后的代码块
15
16 # 条件表达式恒为True的循环语句，称为死循环，它会一直运行，慎用！
17 # while True :
18 #     print('hello')
19
20 # 循环的三个要件（表达式）
21 # 初始化表达式，通过初始化表达式初始化一个变量
22 # i = 0
23
24 # # 条件表达式，条件表达式用来设置循环执行的条件
25 # while i < 10 :
26 #     print(i)
27 #     # 更新表达式，修改初始化变量的值
28 #     i += 1
29
30 # 创建一个执行十次的循环
31 i = 0
32 while i < 10 :
33     i += 1
34     print(i, 'hello')
35 else :
36     print('else中的代码块')
```

break&continue

```
1 # break
2 # break可以用来立即退出循环语句（包括else）
3 # continue
4 # continue可以用来跳过当次循环
5 # break和continue都是只对离他最近的循环起作用
6 # pass
7 # pass是用来在判断或循环语句中占位的
8
9 # 创建一个5次的循环
10 # i = 0
11 # while i < 5:
12 #     if i == 3:
13 #         break
14 #     print(i)
15 #     i += 1
16 # else :
17 #     print('循环结束')
18
19 # i = 0
20 # while i < 5:
21 #     i += 1
22 #     if i == 2:
23 #         continue
24 #     print(i)
25 # else :
26 #     print('循环结束')
27
28 i = 0
29 if i < 5:
30     pass
```

序列

列表

简介

```
1 # 创建列表，通过[]来创建列表
2 my_list = [] # 创建了一个空列表
3 # print(my_list , type(my_list))
4
5 # 列表存储的数据，我们称为元素
6 # 一个列表中可以存储多个元素，也可以在创建列表时，来指定列表中的元素
7 my_list = [10] # 创建一个只包含一个元素的列表
8
9 # 当向列表中添加多个元素时，多个元素之间使用,隔开
10 my_list = [10,20,30,40,50] # 创建了一个保护有5个元素的列表
11
12 # 列表中可以保存任意的对象
13 my_list = [10,'hello',True,None,[1,2,3],print]
14
15 # 列表中的对象都会按照插入的顺序存储到列表中，
16 # 第一个插入的对象保存到第一个位置，第二个保存到第二个位置
17 # 我们可以通过索引（index）来获取列表中的元素
18 # 索引是元素在列表中的位置，列表中的每一个元素都有一个索引
19 # 索引是从0开始的整数，列表第一个位置索引为0，第二个位置索引为1，第三个位置索引为2，以此类推
20 my_list = [10,20,30,40,50]
21
22 # 通过索引获取列表中的元素
23 # 语法：my_list[索引] my_list[0]
24 # print(my_list[4])
25 # 如果使用的索引超过了最大的范围，会抛出异常
26 # print(my_list[5]) IndexError: list index out of range
27
28 # 获取列表的长度，列表中元素的个数
29 # len()函数，通过该函数可以获取列表的长度
30 # 获取到的长度的值，是列表的最大索引 + 1
31 print(len(my_list)) # 5
```

切片

```
1 # 切片
2 # 切片指从现有列表中，获取一个子列表
3 # 创建一个列表，一般创建列表时，变量的名字会使用复数
4 stus = ['孙悟空', '猪八戒', '沙和尚', '唐僧', '蜘蛛精', '白骨精']
5
6 # 列表的索引可以是负数
7 # 如果索引是负数，则从后向前获取元素，-1表示倒数第一个，-2表示倒数第二个 以此类推
8 # print(stus[-2])
9
10 # 通过切片来获取指定的元素
11 # 语法：列表[起始:结束]
12 # 通过切片获取元素时，会包括起始位置的元素，不会包括结束位置的元素
13 # 做切片操作时，总会返回一个新的列表，不会影响原来的列表
14 # 起始和结束位置的索引都可以省略不写
15 # 如果省略结束位置，则会一直截取到最后
16 # 如果省略起始位置，则会从第一个元素开始截取
17 # 如果起始位置和结束位置全部省略，则相当于创建了一个列表的副本
18 # print(stus[1:])
19 # print(stus[:3])
20 # print(stus[:])
21 # print(stus)
22
23 # 语法：列表[起始:结束:步长]
24 # 步长表示，每次获取元素的间隔，默认值是1
25 # print(stus[0:5:3])
26 # 步长不能是0，但是可以是负数
27 # print(stus[::0]) ValueError: slice step cannot be zero
28 # 如果是负数，则会从列表的后部向前边取元素
29 print(stus[::-1])
```

通用操作

```
1 # + 和 *
2 # +可以将两个列表拼接为一个列表
3 my_list = [1,2,3] + [4,5,6]
4
5 # * 可以将列表重复指定的次数
6 my_list = [1,2,3] * 5
7
8 # print(my_list)
9
10 # 创建一个列表
11 stus = ['孙悟空','猪八戒','沙和尚','唐僧','蜘蛛精','白骨精','沙和尚','沙和尚']
12
13 # in 和 not in
14 # in用来检查指定元素是否存在于列表中
15 # 如果存在，返回True，否则返回False
16 # not in用来检查指定元素是否不在列表中
17 # 如果不在，返回True，否则返回False
18 # print('牛魔王' not in stus)
19 # print('牛魔王' in stus)
20
21 # len()获取列表中的元素的个数
22
23 # min() 获取列表中的最小值
24 # max() 获取列表中的最大值
25 arr = [10,1,2,5,100,77]
26 # print(min(arr) , max(arr))
27
28 # 两个方法（method），方法和函数基本上是一样，只不过方法必须通过 对象.方法() 的形式调用
29 # xxx.print() 方法实际上就是和对象关系紧密的函数
30 # s.index() 获取指定元素在列表中的第一次出现时索引
31 # print(stus.index('沙和尚'))
32 # index()的第二个参数，表示查找的起始位置 ， 第三个参数，表示查找的结束位置
33 # print(stus.index('沙和尚',3,7))
34 # 如果要获取列表中没有的元素，会抛出异常
35 # print(stus.index('牛魔王')) ValueError: '牛魔王' is not in list
36 # s.count() 统计指定元素在列表中出现的次数
37 print(stus.count('牛魔王'))
```

修改元素

```
1 # 创建一个列表
2 stus = ['孙悟空', '猪八戒', '沙和尚', '唐僧', '蜘蛛精', '白骨精']
3
4 # print("修改前: ", stus)
5 # 修改列表中的元素
6 # 直接通过索引来修改元素
7 stus[0] = 'sunwukong'
8 stus[2] = '哈哈'
9 # 通过del来删除元素
10 del stus[2] # 删除索引为2的元素
11
12 # print('修改后: ', stus)
13
14 stus = ['孙悟空', '猪八戒', '沙和尚', '唐僧', '蜘蛛精', '白骨精']
15
16 # print("修改前: ", stus)
17
18 # 通过切片来修改列表
19 # 在给切片进行赋值时, 只能使用序列
20 # stus[0:2] = ['牛魔王', '红孩儿'] 使用新的元素替换旧元素
21 # stus[0:2] = ['牛魔王', '红孩儿', '二郎神']
22 # stus[0:0] = ['牛魔王'] # 向索引为0的位置插入元素
23 # 当设置了步长时, 序列中元素的个数必须和切片中元素的个数一致
24 # stus[::2] = ['牛魔王', '红孩儿', '二郎神']
25
26 # 通过切片来删除元素
27 # del stus[0:2]
28 # del stus[::2]
29 # stus[1:3] = []
30
31 # print('修改后: ', stus)
32
33 # 以上操作, 只适用于可变序列
34 s = 'hello'
35 # s[1] = 'a' 不可变序列, 无法通过索引来修改
36 # 可以通过 list() 函数将其他的序列转换为list
37 s = list(s)
```

```
38 print(s)
```

列表方法

```
1 # 列表的方法
2 stus = ['孙悟空','猪八戒','沙和尚','唐僧']
3 # print('原列表: ',stus)
4
5 # append()
6 # 向列表的最后添加一个元素
7 # stus.append('唐僧')
8
9 # insert()
10 # 向列表的指定位置插入一个元素
11 # 参数:
12 #     1.要插入的位置
13 #     2.要插入的元素
14 # stus.insert(2,'唐僧')
15
16 # extend()
17 # 使用新的序列来扩展当前序列
18 # 需要一个序列作为参数, 它会将该序列中的元素添加到当前列表中
19 # stus.extend(['唐僧','白骨精'])
20 # stus += ['唐僧','白骨精']
21
22 # clear()
23 # 清空序列
24 # stus.clear()
25
26 # pop()
27 # 根据索引删除并返回被删除的元素
28
29 # result = stus.pop(2) # 删除索引为2的元素
30 # result = stus.pop() # 删除最后一个
31 # print('result =',result)
32
33 # remove()
34 # 删除指定值得元素, 如果相同值得元素有多个, 只会删除第一个
35 # stus.remove('猪八戒')
36
37 # reverse()
38 # 用来反转列表
```

```
39 # stus.reverse()
40
41 # sort()
42 # 用来对列表中的元素进行排序，默认是升序排列
43 # 如果需要降序排列，则需要传递一个reverse=True作为参数
44 my_list = list('asnbdnbasdabd')
45 my_list = [10,1,20,3,4,5,0,-2]
46
47 print('修改前',my_list)
48
49 my_list.sort(reverse=True)
50 print('修改后',my_list)
51 # print('修改后: ',stus)
```

遍历列表


```
1 # 遍历列表，指的就是将列表中的所有元素取出来
2 # 创建列表
3 stus = ['孙悟空', '猪八戒', '沙和尚', '唐僧', '白骨精', '蜘蛛精']
4
5 # 遍历列表
6 # print(stus[0])
7 # print(stus[1])
8 # print(stus[2])
9 # print(stus[3])
10
11 # 通过while循环来遍历列表
12 # i = 0
13 # while i < len(stus):
14 #     print(stus[i])
15 #     i += 1
16
17 # 通过for循环来遍历列表
18 # 语法：
19 # for 变量 in 序列 :
20 #     代码块
21 # for循环的代码块会执行多次，序列中有几个元素就会执行几次
22 # 没执行一次就会将序列中的一个元素赋值给变量，
23 # 所以我们可以通过变量，来获取列表中的元素
24
25 for s in stus :
26     print(s)
```

使用zip()同时遍历多个列表

```
1 names = ["Alice", "Bob", "Charlie"]
2 ages = [25, 30, 35]
3 for name, age in zip(names, ages):
4     print(name, age)
```

推导

```
1 fizz_buzz_list = [  
2     "FizzBuzz" if i % 15 == 0 else "Fizz" if i % 3 == 0 else "Buzz" if i % 5 == 0 else  
   i for i in range(1, 101)  
3 ]  
4 print(fizz_buzz_list)
```

itertools模块

```
1 import itertools  
2  
3 for combination in itertools.combinations([1, 2, 3], 2):  
4     print(combination)
```

range

```
1 # range()是一个函数，可以用来生成一个自然数的序列
2 r = range(5) # 生成一个这样的序列[0,1,2,3,4]
3 r = range(0,10,2)
4 r = range(10,0,-1)
5 # 该函数需要三个参数
6 #     1.起始位置（可以省略，默认是0）
7 #     2.结束位置
8 #     3.步长（可以省略，默认是1）
9
10 # print(list(r))
11
12 # 通过range()可以创建一个执行指定次数的for循环
13 # for()循环除了创建方式以外，其余的都和while一样，
14 #     包括else、包括break continue都可以在for循环中使用
15 #     并且for循环使用也更加简单
16 # 将之前使用while循环做的练习，再使用for循环完成一次！
17 for i in range(30):
18     print(i)
19
20 # for s in 'hello':
21 #     print(s)
```

元组

```
1 # 元组 tuple
2 # 元组是一个不可变的序列
3 # 它的操作的方式基本上和列表是一致的
4 # 所以你在操作元组时，就把元组当成是一个不可变的列表就ok了
5 # 一般当我们希望数据不改变时，就使用元组，其余情况都使用列表
6
7 # 创建元组
8 # 使用()来创建元组
9 my_tuple = () # 创建了一个空元组
10 # print(my_tuple,type(my_tuple)) # <class 'tuple'>
11
12 my_tuple = (1,2,3,4,5) # 创建了一个5个元素的元组
13 # 元组是不可变对象，不能尝试为元组中的元素重新赋值
14 # my_tuple[3] = 10 TypeError: 'tuple' object does not support item assignment
15 # print(my_tuple[3])
16
17 # 当元组不是空元组时，括号可以省略
18 # 如果元组不是空元组，它里边至少要有有一个，
19 my_tuple = 10,20,30,40
20 my_tuple = 40,
21 # print(my_tuple , type(my_tuple))
22
23 my_tuple = 10 , 20 , 30 , 40
24
25 # 元组的解包（解构）
26 # 解包指就是将元组当中每一个元素都赋值给一个变量
27 a,b,c,d = my_tuple
28
29 # print("a =",a)
30 # print("b =",b)
31 # print("c =",c)
32 # print("d =",d)
33
34 a = 100
35 b = 300
36 # print(a , b)
37
38 # 交互a 和 b的值，这时我们就可以利用元组的解包
```

```
39 a , b = b , a
40
41 # print(a , b)
42 my_tuple = 10 , 20 , 30 , 40
43
44
45 # 在对一个元组进行解包时，变量的数量必须和元组中的元素的数量一致
46 # 也可以在变量前边添加一个*，这样变量将会获取元组中所有剩余的元素
47 a , b , *c = my_tuple
48 a , *b , c = my_tuple
49 *a , b , c = my_tuple
50 a , b , *c = [1,2,3,4,5,6,7]
51 a , b , *c = 'hello world'
52 # 不能同时出现两个或以上的*变量
53 # *a , *b , c = my_tuple SyntaxError: two starred expressions in assignment
54 print('a =',a)
55 print('b =',b)
56 print('c =',c)
57
58
```

字典

```
1 # 字典
2 # 使用 {} 来创建字典
3 d = {} # 创建了一个空字典
4
5 # 创建一个保护有数据的字典
6 # 语法:
7 # {key:value,key:value,key:value}
8 # 字典的值可以是任意对象
9 # 字典的键可以是任意的不可变对象(int、str、bool、tuple ...), 但是一般我们都会使用str
10 # 字典的键是不能重复的, 如果出现重复的后边的会替换到前边的
11 # d = {'name':'孙悟空' , 'age':18 , 'gender':'男' , 'name':'sunwukong'}
12 d = {
13     'name':'孙悟空' ,
14     'age':18 ,
15     'gender':'男' ,
16     'name':'sunwukong'
17 }
18
19 # print(d , type(d))
20
21 # 需要根据键来获取值
22 # print(d['name'],d['age'],d['gender'])
23
24 # 如果使用了字典中不存在的键, 会报错
25 # print(d['hello']) KeyError: 'hello'
```

```
1 # 创建字典
2 # 使用{}
3 # 语法: {k1:v1,k2:v2,k3:v3}
4
5 # 使用 dict()函数来创建字典
6 # 每一个参数都是一个键值对, 参数名就是键, 参数名就是值 (这种方式创建的字典, key都是字符串)
7 d = dict(name='孙悟空',age=18,gender='男')
8
9 # 也可以将一个包含有双值子序列的序列转换为字典
10 # 双值序列, 序列中只有两个值, [1,2] ('a',3) 'ab'
11 # 子序列, 如果序列中的元素也是序列, 那么我们就称这个元素为子序列
12 # [(1,2),(3,5)]
13 d = dict([( 'name', '孙悟空'), ('age', 18)])
14 # print(d , type(d))
15 d = dict(name='孙悟空',age=18,gender='男')
16
17 # len() 获取字典中键值对的个数
18 # print(len(d))
19
20 # in 检查字典中是否包含指定的键
21 # not in 检查字典中是否不包含指定的键
22 # print('hello' in d)
23
24 # 获取字典中的值, 根据键来获取值
25 # 语法: d[key]
26 # print(d['age'])
27
28 # n = 'name'
29 # print(d[n])
30
31 # 通过[]来获取值时, 如果键不存在, 会抛出异常 KeyError
32 # get(key[, default]) 该方法用来根据键来获取字典中的值
33 # 如果获取的键在字典中不存在, 会返回None
34 # 也可以指定一个默认值, 来作为第二个参数, 这样获取不到值时将会返回默认值
35 # print(d.get('name'))
36 # print(d.get('hello','默认值'))
37
38 # 修改字典
```

```
39 # d[key] = value 如果key存在则覆盖，不存在则添加
40 d['name'] = 'sunwukong' # 修改字典的key-value
41 d['address'] = '花果山' # 向字典中添加key-value
42
43 # print(d)
44 #.setdefault(key[, default]) 可以用来向字典中添加key-value
45 # 如果key已经存在于字典中，则返回key的值，不会对字典做任何操作
46 # 如果key不存在，则向字典中添加这个key，并设置value
47 result = d.setdefault('name','猪八戒')
48 result = d.setdefault('hello','猪八戒')
49
50 # print('result =',result)
51 # print(d)
52
53 # update([other])
54 # 将其他的字典中的key-value添加到当前字典中
55 # 如果有重复的key，则后边的会替换到当前的
56 d = {'a':1,'b':2,'c':3}
57 d2 = {'d':4,'e':5,'f':6, 'a':7}
58 d.update(d2)
59
60 # print(d)
61 # 删除，可以使用 del 来删除字典中的 key-value
62 del d['a']
63 del d['b']
64
65 # popitem()
66 # 随机删除字典中的一个键值对，一般都会删除最后一个键值对
67 # 删除之后，它会将删除的key-value作为返回值返回
68 # 返回的是一个元组，元组中有两个元素，第一个元素是删除的key，第二个是删除的value
69 # 当使用popitem()删除一个空字典时，会抛出异常 KeyError: 'popitem(): dictionary is empty'
70 # d.popitem()
71 # result = d.popitem()
72
73 # pop(key[, default])
74 # 根据key删除字典中的key-value
75 # 将会被删除的value返回！
76 # 如果删除不存在的key，会抛出异常
77 # 如果指定了默认值，再删除不存在的key时，不会报错，而是直接返回默认值
```



```

78 result = d.pop('d')
79 result = d.pop('z','这是默认值')
80
81 # del d['z'] z不存在，报错
82 # result = d.popitem()
83 # result = d.popitem()
84 # result = d.popitem()
85 # result = d.popitem()
86
87 # clear()用来清空字典
88 d.clear()
89
90 # print('result =',result)
91 # print(d)
92
93 # copy()
94 # 该方法用于对字典进行浅复制
95 # 复制以后的对象，和原对象是独立，修改一个不会影响另一个
96 # 注意，浅复制会简单复制对象内部的值，如果值也是一个可变对象，这个可变对象不会被复制
97 d = {'a':1,'b':2,'c':3}
98 d2 = d.copy()
99 # d['a'] = 100
100
101 d = {'a':{'name':'孙悟空','age':18},'b':2,'c':3}
102 d2 = d.copy()
103 d2['a']['name'] = '猪八戒'
104
105
106 print('d = ',d , id(d))
107 print('d2 = ',d2 , id(d2))

```

字典遍历

```
1 # 遍历字典
2 # keys() 该方法会返回字典的所有的key
3 # 该方法会返回一个序列，序列中保存有字典的所有的键
4 d = {'name': '孙悟空', 'age': 18, 'gender': '男'}
5
6 # 通过遍历keys()来获取所有的键
7 # for k in d.keys() :
8 #     print(k , d[k])
9
10 # values()
11 # 该方法会返回一个序列，序列中保存有字典的左右的值
12 # for v in d.values():
13 #     print(v)
14
15 # items()
16 # 该方法会返回字典中所有的项
17 # 它会返回一个序列，序列中包含有双值子序列
18 # 双值分别是，字典中的key和value
19 # print(d.items())
20 for k,v in d.items() :
21     print(k , '=' , v)
```

集合

```
1 # 集合
2 # 使用 {} 来创建集合
3 s = {10,3,5,1,2,1,2,3,1,1,1,1} # <class 'set'>
4 # s = {[1,2,3],[4,6,7]} TypeError: unhashable type: 'list'
5 # 使用 set() 函数来创建集合
6 s = set() # 空集合
7 # 可以通过set()来将序列和字典转换为集合
8 s = set([1,2,3,4,5,1,1,2,3,4,5])
9 s = set('hello')
10 s = set({'a':1,'b':2,'c':3}) # 使用set()将字典转换为集合时，只会包含字典中的键
11
12 # 创建集合
13 s = {'a' , 'b' , 1 , 2 , 3 , 1}
14
15 # 使用in和not in来检查集合中的元素
16 print('c' in s)
17
18 # 使用len()来获取集合中元素的数量
19 print(len(s))
20
21 # add() 向集合中添加元素
22 s.add(10)
23 s.add(30)
24
25 # update() 将一个集合中的元素添加到当前集合中
26 # update()可以传递序列或字典作为参数，字典只会使用键
27 s2 = set('hello')
28 s.update(s2)
29 s.update((10,20,30,40,50))
30 s.update({'10':'ab',20:'bc',100:'cd',1000:'ef'})
31
32 # {1, 2, 3, 100, 40, 'o', 10, 1000, 'a', 'h', 'b', 'l', 20, 50, 'e', 30}
33 # pop()随机删除并返回一个集合中的元素
34 result = s.pop()
35
36 #remove()删除集合中的指定元素
37 s.remove(100)
38 s.remove(1000)
```

```
39
40 # clear()清空集合
41 s.clear()
42
43 # copy()对集合进行浅复制
44
45 # print(result)
46 print(s , type(s))
```

```

1 # 在对集合做运算时，不会影响原来的集合，而是返回一个运算结果
2 # 创建两个集合
3 s = {1,2,3,4,5}
4 s2 = {3,4,5,6,7}
5
6 # & 交集运算
7 result = s & s2 # {3, 4, 5}
8
9 # | 并集运算
10 result = s | s2 # {1,2,3,4,5,6,7}
11
12 # - 差集
13 result = s - s2 # {1, 2}
14
15 # ^ 异或集 获取只在一个集合中出现的元素
16 result = s ^ s2 # {1, 2, 6, 7}
17
18 # <= 检查一个集合是否是另一个集合的子集
19 # 如果a集合中的元素全部都在b集合中出现，那么a集合就是b集合的子集，b集合是a集合超集
20 a = {1,2,3}
21 b = {1,2,3,4,5}
22
23 result = a <= b # True
24 result = {1,2,3} <= {1,2,3} # True
25 result = {1,2,3,4,5} <= {1,2,3} # False
26
27 # < 检查一个集合是否是另一个集合的真子集
28 # 如果超集b中含有子集a中所有元素，并且b中还有a中没有的元素，则b就是a的真超集，a是b的真子集
29 result = {1,2,3} < {1,2,3} # False
30 result = {1,2,3} < {1,2,3,4,5} # True
31
32 # >= 检查一个集合是否是另一个的超集
33 # > 检查一个集合是否是另一个的真超集
34 print('result =',result)

```

集合去重

```
1 duplicates = [1, 2, 2, 3, 4, 4, 5]
2 unique_list = list(set(duplicates))
3 print(unique_list)
```

可变对象

```
1 # 可变对象
2 a = [1,2,3]
3 # print('修改前: ', a , id(a))
4
5 # 通过索引修改列表
6 a[0] = 10
7 # print('修改后: ', a , id(a))
8
9 #为变量重新赋值
10 a = [4,5,6]
11 print('修改后: ', a , id(a))
12
13
14 a = [1,2,3]
15 b = a
16 # b[0] = 10
17 b = [10,2,3]
18 # print("a",a,id(a))
19 # print("b",b,id(b))
20
21 # == , != , is , is not
22 # ==, != 比较的是对象的值是否相等
23 # is , is not 比较的是对象的id是否相等（比较两个对象是否是同一个对象）
24
25 a = [1,2,3]
26 b = [1,2,3]
27 print(a,b)
28 print(id(a),id(b))
29 print(a == b) # a和b的值相等，使用==会返回True
30 print(a is b) # a和b不是同一个对象，内存地址不同，使用is会返回False
```

函数

函数的定义

```
1 # 定义一个函数
2 def fn() :
3     print('这是我的第一个函数! ')
4     print('hello')
5     print('今天天气真不错! ')
```

函数信息

```
1 # 打印fn
2 print(fn) <function fn at 0x03D2B618>
3 print(type(fn)) <class 'function'>
```

带参函数

```
1 # 定义函数时指定形参
2 def fn2(a , b) :
3     # print('a =',a)
4     # print('b =',b)
5     print(a,"+",b,"=",a + b)
```

带参函数调用

```
1 # 调用函数时，来传递实参
2 fn2(10,20)
3 fn2(123,456)
```

参数默认值


```
1 # 定义一个函数
2 # 定义形参时，可以为形参指定默认值
3 # 指定了默认值以后，如果用户传递了参数则默认值没有任何作用
4 # 如果用户没有传递，则默认值就会生效
5 def fn(a = 5 , b = 10 , c = 20):
6     print('a =',a)
7     print('b =',b)
8     print('c =',c)
9
10 #调用
11 # fn(1 , 2 , 3)
12 # fn(1 , 2)
13 # fn()
```

关键字参数

```
1 # 关键字参数
2 # 关键字参数，可以不按照形参定义的顺序去传递，而直接根据参数名去传递参数
3 fn(b=1 , c=2 , a=3)
4     print('hello' , end='')
5 # 位置参数和关键字参数可以混合使用
6 # 混合使用关键字和位置参数时，必须将位置参数写到前面
7 fn(1,c=30)
```

不定长参数

```
1 # 不定长的参数
2 # 定义一个函数，可以求任意个数字的和
3 def sum(*nums):
4     # 定义一个变量，来保存结果
5     result = 0
6     # 遍历元组，并将元组中的数进行累加
7     for n in nums :
8         result += n
9     print(result)
10
11 #调用
12 sum(123,456,789,10,20,30,40)
13
14 # 在定义函数时，可以在形参前边加上一个*，这样这个形参将会获取到所有的实参
15 # 它将会将所有的实参保存到一个元组中
16 # a,b,*c = (1,2,3,4,5,6)
17
18 # *a会接受所有的位置实参，并且会将这些实参统一保存到一个元组中（装包）
19 def fn(*a):
20     print("a =",a,type(a))
21
22 # fn(1,2,3,4,5)
23 # 带星号的形参只能有一个
24 # 带星号的参数，可以和其他参数配合使用
25 # 第一个参数给a，第二个参数给b，剩下的都保存到c的元组中
26 # def fn2(a,b,*c):
27 #     print('a =',a)
28 #     print('b =',b)
29 #     print('c =',c)
30
31 # 可变参数不是必须写在最后，但是注意，带*的参数后的所有参数，必须以关键字参数的形式传递
32 # 第一个参数给a，剩下的位置参数给b的元组，c必须使用关键字参数
33 # def fn2(a,*b,c):
34 #     print('a =',a)
35 #     print('b =',b)
36 #     print('c =',c)
37
38 # 所有的位置参数都给a，b和c必须使用关键字参数
```

```
39 # def fn2(*a,b,c):
40 #     print('a =',a)
41 #     print('b =',b)
42 #     print('c =',c)
43
44 # 如果在形参的开头直接写一个*,则要求我们的所有的参数必须以关键字参数的形式传递
45 def fn2(*,a,b,c):
46     print('a =',a)
47     print('b =',b)
48     print('c =',c)
49 # fn2(a=3,b=4,c=5)
50
51 # *形参只能接收位置参数,而不能接收关键字参数
52 # def fn3(*a) :
53 #     print('a =',a)
54
55 # **形参可以接收其他的关键字参数,它会将这些参数统一保存到一个字典中
56 # 字典的key就是参数的名字,字典的value就是参数的值
57 # **形参只能有一个,并且必须写在所有参数的最后
58 def fn3(b,c,**a) :
59     print('a =',a,type(a))
60     print('b =',b)
61     print('c =',c)
62
63 # fn3(b=1,d=2,c=3,e=10,f=20)
64
65 # 参数的解包(拆包)
66 def fn4(a,b,c):
67     print('a =',a)
68     print('b =',b)
69     print('c =',c)
70
71 # 创建一个元组
72 t = (10,20,30)
73
74 # 传递实参时,也可以在序列类型的参数前添加星号,这样他会自动将序列中的元素依次作为参数传递
75 # 这里要求序列中元素的个数必须和形参的个数的一致
76 # fn4(*t)
77
```

```
78 # 创建一个字典
79 d = {'a':100, 'b':200, 'c':300}
80 # 通过 **来对一个字典进行解包操作
81 fn4(**d)
```

返回值

```
1 # 返回值，返回值就是函数执行以后返回的结果
2 # 可以通过 return 来指定函数的返回值
3 # 可以之间使用函数的返回值，也可以通过一个变量来接收函数的返回值
4
5 def sum(*nums):
6     # 定义一个变量，来保存结果
7     result = 0
8     # 遍历元组，并将元组中的数进行累加
9     for n in nums :
10         result += n
11     print(result)
12
13 # sum(123,456,789)
14
15
16 # return 后边跟什么值，函数就会返回什么值
17 # return 后边可以跟任意的对象，返回值甚至可以是一个函数
18 def fn():
19     # return 'Hello'
20     # return [1,2,3]
21     # return {'k':'v'}
22     def fn2() :
23         print('hello')
24
25     return fn2 # 返回值也可以是一个函数
26
27 r = fn() # 这个函数的执行结果就是它的返回值
28 # r()
29 # print(fn())
30 # print(r)
31
32 # 如果仅仅写一个return 或者 不写return，则相当于return None
33 def fn2() :
34     a = 10
35     return
36
37 # 在函数中，return后的代码都不会执行，return 一旦执行函数自动结束
38 def fn3():
```

```
39     print('hello')
40     return
41     print('abc')
42
43 # r = fn3()
44 # print(r)
45
46 def fn4() :
47     for i in range(5):
48         if i == 3 :
49             # break 用来退出当前循环
50             # continue 用来跳过当次循环
51             return # return 用来结束函数
52         print(i)
53     print('循环执行完毕! ')
54
55 # fn4()
56
57 def sum(*nums):
58     # 定义一个变量，来保存结果
59     result = 0
60     # 遍历元组，并将元组中的数进行累加
61     for n in nums :
62         result += n
63     return result
64
65 r = sum(123,456,789)
66
67 # print(r + 778)
68
69 def fn5():
70     return 10
71
72 # fn5 和 fn5()的区别
73 print(fn5) # fn5是函数对象，打印fn5实际是在打印函数对象 <function fn5 at 0x05771BB8>
74 print(fn5()) # fn5()是在调用函数，打印fn5()实际上是在打印fn5()函数的返回值 10
75
```

作用域与命名空间

```
1 # 作用域 (scope)
2 # 作用域指的是变量生效的区域
3 b = 20 # 全局变量
4
5 def fn():
6     a = 10 # a定义在了函数内部，所以他的作用域就是函数内部，函数外部无法访问
7     print('函数内部: ', 'a =', a)
8     print('函数内部: ', 'b =', b)
9
10 # fn()
11
12
13 # print('函数外部: ', 'a =', a)
14 # print('函数外部: ', 'b =', b)
15
16 # 在Python中一共有两种作用域
17 # 全局作用域
18 # - 全局作用域在程序执行时创建，在程序执行结束时销毁
19 # - 所有函数以外的区域都是全局作用域
20 # - 在全局作用域中定义的变量，都属于全局变量，全局变量可以在程序的任意位置被访问
21 #
22 # 函数作用域
23 # - 函数作用域在函数调用时创建，在调用结束时销毁
24 # - 函数每调用一次就会产生一个新的函数作用域
25 # - 在函数作用域中定义的变量，都是局部变量，它只能在函数内部被访问
26 #
27 # 变量的查找
28 # - 当我们使用变量时，会优先在当前作用域中寻找该变量，如果有则使用，
29 #     如果没有则继续去上一级作用域中寻找，如果有则使用，
30 #     如果依然没有则继续去上一级作用域中寻找，以此类推
31 #     直到找到全局作用域，依然没有找到，则会抛出异常
32 #         NameError: name 'a' is not defined
33
34 def fn2():
35     def fn3():
36         print('fn3中:', 'a =', a)
37     fn3()
38
```



```
39 # fn2()
40
41 a = 20
42
43 def fn3():
44     # a = 10 # 在函数中为变量赋值时，默认都是为局部变量赋值
45     # 如果希望在函数内部修改全局变量，则需要使用global关键字，来声明变量
46     global a # 声明在函数内部的使用a是全局变量，此时再去修改a时，就是在修改全局的a
47     a = 10 # 修改全局变量
48     print('函数内部: ', 'a = ', a)
49
50 # fn3()
51 # print('函数外部: ', 'a = ', a)
52
53
54 # 命名空间（namespace）
55 # 命名空间指的是变量存储的位置，每一个变量都需要存储到指定的命名空间当中
56 # 每一个作用域都会有一个它对应的命名空间
57 # 全局命名空间，用来保存全局变量。函数命名空间用来保存函数中的变量
58 # 命名空间实际上就是一个字典，是一个专门用来存储变量的字典
59
60 # locals()用来获取当前作用域的命名空间
61 # 如果在全局作用域中调用locals()则获取全局命名空间，如果在函数作用域中调用locals()则获取函数命名空间
62 # 返回的是一个字典
63 scope = locals() # 当前命名空间
64 print(type(scope))
65 # print(a)
66 # print(scope['a'])
67 # 向scope中添加一个key-value
68 scope['c'] = 1000 # 向字典中添加key-value就相当于在全局中创建了一个变量（一般不建议这么做）
69 # print(c)
70
71 def fn4():
72     a = 10
73     # scope = locals() # 在函数内部调用locals()会获取到函数的命名空间
74     # scope['b'] = 20 # 可以通过scope来操作函数的命名空间，但是也是不建议这么做
75
76     # globals() 函数可以用来在任意位置获取全局命名空间
```

```
77     global_scope = globals()
78     # print(global_scope['a'])
79     global_scope['a'] = 30
80     # print(scope)
81
82     fn4()
83
84
```

递归

```
1 # 尝试求10的阶乘 (10!)
2 # 1! = 1
3 # 2! = 1*2 = 2
4 # 3! = 1*2*3 = 6
5 # 4! = 1*2*3*4 = 24
6
7 # print(1*2*3*4*5*6*7*8*9*10)
8
9 # 创建一个变量保存结果
10 # n = 10
11 # for i in range(1,10):
12 #     n *= i
13
14 # print(n)
15
16 # 创建一个函数，可以用来求任意数的阶乘
17 def factorial(n):
18     '''
19         该函数用来求任意数的阶乘
20
21         参数:
22             n 要求阶乘的数字
23     '''
24
25     # 创建一个变量，来保存结果
26     result = n
27
28     for i in range(1,n):
29         result *= i
30
31     return result
32
33 # 求10的阶乘
34 # print(factorial(20))
35
36 # 递归式的函数
37 # 从前有座山，山里有座庙，庙里有个老和尚讲故事，讲的什么故事呢？
```

```
38 # 从前有座山，山里有座庙，庙里有个老和尚讲故事，讲的什么故事呢？....
39 # 递归简单理解就是自己去引用自己！
40 # 递归式函数，在函数中自己调用自己！
41
42 # 无穷递归，如果这个函数被调用，程序的内存会溢出，效果类似于死循环
43 # def fn():
44 #     fn()
45 # fn()
46
47 # 递归是解决问题的一种方式，它和循环很像
48 # 它的整体思想是，将一个大问题分解为一个个的小问题，直到问题无法分解时，再去解决问题
49 # 递归式函数的两个要件
50 # 1.基线条件
51 #     - 问题可以被分解为的最小问题，当满足基线条件时，递归就不再执行了
52 # 2.递归条件
53 #     - 将问题继续分解的条件
54 # 递归和循环类似，基本是可以互相代替的，
55 # 循环编写起来比较容易，阅读起来稍难
56 # 递归编写起来难，但是方便阅读
57 # 10! = 10 * 9!
58 # 9! = 9 * 8!
59 # 8! = 8 * 7!
60 # ...
61 # 1! = 1
62
63 def factorial(n):
64     '''
65     该函数用来求任意数的阶乘
66
67     参数：
68     n 要求阶乘的数字
69     '''
70     # 基线条件 判断n是否为1，如果为1则此时不能再继续递归
71     if n == 1 :
72         # 1的阶乘就是1，直接返回1
73         return 1
74
75     # 递归条件
76     return n * factorial(n-1)
```

```

77
78 # print(factorial(10))
79
80 # 练习
81 # 创建一个函数 power 来为任意数字做幂运算 n ** i
82 # 10 ** 5 = 10 * 10 ** 4
83 # 10 ** 4 = 10 * 10 ** 3
84 # ...
85 # 10 ** 1 = 10
86 def power(n , i):
87     '''
88         power()用来为任意的数字做幂运算
89
90         参数:
91             n 要做幂运算的数字
92             i 做幂运算的次数
93     '''
94     # 基线条件
95     if i == 1:
96         # 求1次幂
97         return n
98     # 递归条件
99     return n * power(n , i-1)
100
101 # print(power(8,6))
102
103
104
105 #
106 # 练习
107 # 创建一个函数，用来检查一个任意的字符串是否是回文字符串，如果是返回True，否则返回False
108 # 回文字符串，字符串从前往后念和从后往前念是一样的
109 # abcba
110 # abcdefgfedcba
111 # 先检查第一个字符和最后一个字符是否一致，如果不一致则不是回文字符串
112 # 如果一致，则看剩余的部分是否是回文字符串
113 # 检查 abcdefgfedcba 是不是回文
114 # 检查 bcdefgfedcb 是不是回文
115 # 检查 cdefgfedc 是不是回文

```

```
116 # 检查 defgfed 是不是回文
117 # 检查 efgfe 是不是回文
118 # 检查 fgf 是不是回文
119 # 检查 g 是不是回文
120
121 def hui_wen(s):
122     '''
123         该函数用来检查指定的字符串是否回文字符串，如果是返回True，否则返回False
124
125         参数：
126             s: 就是要检查的字符串
127     '''
128     # 基线条件
129     if len(s) < 2 :
130         # 字符串的长度小于2，则字符串一定是回文
131         return True
132     elif s[0] != s[-1]:
133         # 第一个字符和最后一个字符不相等，不是回文字符串
134         return False
135     # 递归条件
136     return hui_wen(s[1:-1])
137
138 # def hui_wen(s):
139 #     '''
140 #         该函数用来检查指定的字符串是否回文字符串，如果是返回True，否则返回False
141 #
142 #         参数：
143 #             s: 就是要检查的字符串
144 #     '''
145 #     # 基线条件
146 #     if len(s) < 2 :
147 #         # 字符串的长度小于2，则字符串一定是回文
148 #         return True
149 #     # 递归条件
150 #     return s[0] == s[-1] and hui_wen(s[1:-1])
151
152 print(hui_wen('abcdefgfedcba'))
153
```

高阶函数

```
1 # 高阶函数
2 # 接收函数作为参数，或者将函数作为返回值的函数是高阶函数
3 # 当我们使用一个函数作为参数时，实际上是将指定的代码传递进了目标函数
4
5 # 创建一个列表
6 l = [1,2,3,4,5,6,7,8,9,10]
7
8 # 定义一个函数
9 # 可以将指定列表中的所有的偶数，保存到一个新的列表中返回
10
11 # 定义一个函数，用来检查一个任意的数字是否是偶数
12 def fn2(i) :
13     if i % 2 == 0 :
14         return True
15
16     return False
17
18 # 这个函数用来检查指定的数字是否大于5
19 def fn3(i):
20     if i > 5 :
21         return True
22     return False
23
24 def fn(func , lst) :
25
26     '''
27         fn()函数可以将指定列表中的所有偶数获取出来，并保存到一个新列表中返回
28
29         参数：
30             lst: 要进行筛选的列表
31     '''
32     # 创建一个新列表
33     new_list = []
34
35     # 对列表进行筛选
36     for n in lst :
37         # 判断n的奇偶
```



```
38         if func(n) :
39             new_list.append(n)
40         # if n > 5 :
41         #     new_list.append(n)
42
43
44
45
46     # 返回新列表
47     return new_list
48
49 # def fn4(i):
50 #     if i % 3 == 0:
51 #         return True
52 #     return False
53
54 def fn4(i):
55     return i % 3 == 0
56
57 # print(fn(fn4 , 1))
58
59 # filter()
60 # filter()可以从序列中过滤出符合条件的元素，保存到一个新的序列中
61 # 参数：
62 # 1.函数，根据该函数来过滤序列（可迭代的结构）
63 # 2.需要过滤的序列（可迭代的结构）
64 # 返回值：
65 # 过滤后的新序列（可迭代的结构）
66
67 # fn4是作为参数传递进filter()函数中
68 # 而fn4实际上只有一个作用，就是作为filter()的参数
69 # filter()调用完毕以后，fn4就已经没用
70 # 匿名函数 lambda 函数表达式 （语法糖）
71 # lambda函数表达式专门用来创建一些简单的函数，他是函数创建的又一种方式
72 # 语法：lambda 参数列表 : 返回值
73 # 匿名函数一般都是作为参数使用，其他地方一般不会使用
74
75 def fn5(a , b):
76     return a + b
```

```
77
78 # (lambda a,b : a + b)(10,20)
79 # 也可以将匿名函数赋值给一个变量，一般不会这么做
80 fn6 = lambda a,b : a + b
81 # print(fn6(10,30))
82
83
84 r = filter(lambda i : i > 5 , l)
85 # print(list(r))
86
87 # map()
88 # map()函数可以对可迭代对象中的所有元素做指定的操作，然后将其添加到一个新的对象中返回
89 l = [1,2,3,4,5,6,7,8,9,10]
90
91 r = map(lambda i : i ** 2 , l)
92
93 # print(list(r))
94
95 # sort()
96 # 该方法用来对列表中的元素进行排序
97 # sort()方法默认是直接比较列表中的元素的大小
98 # 在sort()可以接收一个关键字参数， key
99 # key需要一个函数作为参数，当设置了函数作为参数
100 # 每次都会以列表中的一个元素作为参数来调用函数，并且使用函数的返回值来比较元素的大小
101 l = ['bb', 'aaaa', 'c', 'dddddddddd', 'fff']
102 # l.sort(key=len)
103
104 l = [2,5,'1',3,'6','4']
105 l.sort(key=int)
106 # print(l)
107
108 # sorted()
109 # 这个函数和sort()的用法基本一致，但是sorted()可以对任意的序列进行排序
110 # 并且使用sorted()排序不会影响原来的对象，而是返回一个新对象
111
112 l = [2,5,'1',3,'6','4']
113 # l = "123765816742634781"
114
```

```
115 print('排序前:',l)
116 print(sorted(l,key=int))
117 print('排序后:',l)
```

闭包

```
1 # 将函数作为返回值返回，也是一种高阶函数
2 # 这种高阶函数我们也称为叫做闭包，通过闭包可以创建一些只有当前函数能访问的变量
3 #  可以将一些私有的数据藏到的闭包中
4
5 def fn():
6     a = 10
7     # 函数内部再定义一个函数
8     def inner():
9         print('我是fn2' , a)
10
11     # 将内部函数 inner作为返回值返回
12     return inner
13
14 # r是一个函数，是调用fn()后返回的函数
15 # 这个函数实在fn()内部定义，并不是全局函数
16 # 所以这个函数总是能访问到fn()函数内的变量
17 r = fn()
18
19 # r()
20
21 # 求多个数的平均值
22 # nums = [50,30,20,10,77]
23
24 # sum()用来求一个列表中所有元素的和
25 # print(sum(nums)/len(nums))
26
27 # 形成闭包的要件
28 # ① 函数嵌套
29 # ② 将内部函数作为返回值返回
30 # ③ 内部函数必须要使用到外部函数的变量
31 def make_averager():
32     # 创建一个列表，用来保存数值
33     nums = []
34
35     # 创建一个函数，用来计算平均值
36     def averager(n) :
37         # 将n添加到列表中
38         nums.append(n)
```

```
39         # 求平均值
40         return sum(nums)/len(nums)
41
42     return averager
43
44 averager = make_averager()
45
46 print(averager(10))
47 print(averager(20))
48 print(averager(30))
49 print(averager(40))
```

装饰器

```
1  # 创建几个函数
2
3  def add(a , b):
4      '''
5          求任意两个数的和
6      '''
7      r = a + b
8      return r
9
10
11 def mul(a , b):
12     '''
13         求任意两个数的积
14     '''
15     r = a * b
16     return r
17
18 # 希望函数可以在计算前，打印开始计算，计算结束后打印计算完毕
19 # 我们可以直接通过修改函数中的代码来完成这个需求，但是会产生以下一些问题
20 # ① 如果要修改的函数过多，修改起来会比较麻烦
21 # ② 并且不方便后期的维护
22 # ③ 并且这样做会违反开闭原则（OCP）
23 # 程序的设计，要求开发对程序的扩展，要关闭对程序的修改
24
25
26 # r = add(123,456)
27 # print(r)
28
29 # 我们希望在修改原函数的情况下，来对函数进行扩展
30 def fn():
31     print('我是fn函数....')
32
33 # 只需要根据现有的函数，来创建一个新的函数
34 def fn2():
35     print('函数开始执行~~~')
36     fn()
37     print('函数执行结束~~~')
```

```
38
39 # fn2()
40
41 def new_add(a,b):
42     print('计算开始~~~')
43     r = add(a,b)
44     print('计算结束~~~')
45     return r
46
47 # r = new_add(111,222)
48 # print(r)
49
50 # 上边的方式，已经可以在不修改源代码的情况下对函数进行扩展了
51 # 但是，这种方式要求我们每扩展一个函数就要手动创建一个新的函数，实在是太麻烦了
52 # 为了解决这个问题，我们创建一个函数，让这个函数可以自动的帮助我们生产函数
53
54 def begin_end(old):
55     '''
56         用来对其他函数进行扩展，使其他函数可以在执行前打印开始执行，执行后打印执行结束
57
58         参数：
59             old 要扩展的函数对象
60     '''
61     # 创建一个新函数
62     def new_function(*args , **kwargs):
63         print('开始执行~~~~')
64         # 调用被扩展的函数
65         result = old(*args , **kwargs)
66         print('执行结束~~~~')
67         # 返回函数的执行结果
68         return result
69
70     # 返回新函数
71     return new_function
72
73 f = begin_end(fn)
74 f2 = begin_end(add)
75 f3 = begin_end(mul)
76
```

```

77 # r = f()
78 # r = f2(123,456)
79 # r = f3(123,456)
80 # print(r)
81 # 向begin_end()这种函数我们就称它为装饰器
82 # 通过装饰器，可以在不修改原来函数的情况下来对函数进行扩展
83 # 在开发中，我们都是通过装饰器来扩展函数的功能的
84 # 在定义函数时，可以通过@装饰器，来使用指定的装饰器，来装饰当前的函数
85 # 可以同时为一个函数指定多个装饰器，这样函数将会安装从内向外的顺序被装饰
86
87 def fn3(old):
88     '''
89         用来对其他函数进行扩展，使其他函数可以在执行前打印开始执行，执行后打印执行结束
90
91         参数：
92             old 要扩展的函数对象
93     '''
94     # 创建一个新函数
95     def new_function(*args , **kwargs):
96         print('fn3装饰~开始执行~~~~')
97         # 调用被扩展的函数
98         result = old(*args , **kwargs)
99         print('fn3装饰~执行结束~~~~')
100        # 返回函数的执行结果
101        return result
102
103    # 返回新函数
104    return new_function
105
106 @fn3
107 @begin_end
108 def say_hello():
109     print('大家好~~~')
110
111 say_hello()
112

```


类

简介

```
1  a = int(10) # 创建一个int类的实例
2  b = str('hello') # 创建一个str类的实例
3
4  # print(a , type(a))
5  # print(b , type(b))
6
7  # 定义一个简单的类
8  # 使用class关键字来定义类，语法和函数很像！
9  # class 类名([父类]):
10 #     代码块
11 # <class '__main__.MyClass'>
12 class MyClass():
13     pass
14
15 # print(MyClass)
16 # 使用MyClass创建一个对象
17 # 使用类来创建对象，就像调用一个函数一样
18 mc = MyClass() # mc就是通过MyClass创建的对象，mc是MyClass的实例
19 mc_2 = MyClass()
20 mc_3 = MyClass()
21 mc_4 = MyClass()
22 # mc mc_2 mc_3 mc_4 都是MyClass的实例，他们都是一类对象
23 # isinstance()用来检查一个对象是否是一个类的实例
24 result = isinstance(mc_2,MyClass)
25 result = isinstance(mc_2,str)
26
27 # print(mc , type(mc))
28 # print('result =',result)
29
30 # print(id(MyClass) , type(MyClass))
31
32 # 现在我们通过MyClass这个类创建的对象都是一个空对象
33 # 也就是对象中实际上什么都没有，就相当于是一个空的盒子
34 # 可以向对象中添加变量，对象中的变量称为属性
35 # 语法：对象.属性名 = 属性值
36 mc.name = '孙悟空'
37 mc_2.name = '猪八戒'
38
```

```
39 print(mc_2.name)
```

```
40
```

定义

```
1 # 尝试定义一个表示人的类
2 class Person :
3     # 在类的代码块中，我们可以定义变量和函数
4     # 在类中所定义的变量，将会成为所有的实例的公共属性
5     # 所有实例都可以访问这些变量
6     name = 'swk' # 公共属性，所有实例都可以访问
7
8     # 在类中也可以定义函数，类中定义的函数，我们称为方法
9     # 这些方法可以通过该类的所有实例来访问
10
11     def say_hello(self) :
12         # 方法每次被调用时，解析器都会自动传递第一个实参
13         # 第一个参数，就是调用方法的对象本身，
14         # 如果是p1调的，则第一个参数就是p1对象
15         # 如果是p2调的，则第一个参数就是p2对象
16         # 一般我们都会将这个参数命名为self
17
18         # say_hello()这个方法，可以显示如下格式的数据：
19         # 你好！我是 xxx
20         # 在方法中不能直接访问类中的属性
21         print('你好！我是 %s' %self.name)
22
23 # 创建Person的实例
24 p1 = Person()
25 p2 = Person()
26
27 # print(p2.name)
28
29 # 调用方法，对象.方法名()
30 # 方法调用和函数调用的区别
31 # 如果是函数调用，则调用时传几个参数，就会有几个实参
32 # 但是如果是方法调用，默认传递一个参数，所以方法中至少要定义一个形参
33
34
35 # 修改p1的name属性
36 p1.name = '猪八戒'
37 p2.name = '沙和尚'
38
```

```
39 p1.say_hello() # '你好！我是 猪八戒'
40 p2.say_hello() # '你好！我是 沙和尚'
41
42 # del p2.name # 删除p2的name属性
43
44 # print(p1.name)
45 # print(p2.name)
46
```

对象初始化

```
1 class Person :
2     # 在类中可以定义一些特殊方法（魔术方法）
3     # 特殊方法都是以__开头，__结尾的方法
4     # 特殊方法不需要我们自己调用，不要尝试去调用特殊方法
5     # 特殊方法将会在特殊的时刻自动调用
6     # 学习特殊方法：
7     #     1.特殊方法什么时候调用
8     #     2.特殊方法有什么作用
9     # 创建对象的流程
10    # p1 = Person()的运行流程
11    #     1.创建一个变量
12    #     2.在内存中创建一个新对象
13    #     3.__init__(self)方法执行
14    #     4.将对象的id赋值给变量
15
16    # init会在对象创建以后离开执行
17    # init可以用来向新创建的对象中初始化属性
18    # 调用类创建对象时，类后边的所有参数都会依次传递到init()中
19    def __init__(self,name):
20        # print(self)
21        # 通过self向新建的对象中初始化属性
22        self.name = name
23
24    def say_hello(self):
25        print('大家好，我是%s'%self.name)
26
27
28 # 目前来讲，对于Person类来说name是必须的，并且每一个对象中的name属性基本上都是不同
29 # 而我们现在是将name属性在定义为对象以后，手动添加到对象中，这种方式很容易出现错误
30 # 我们希望，在创建对象时，必须设置name属性，如果不设置对象将无法创建
31 # 并且属性的创建应该是自动完成的，而不是在创建对象以后手动完成
32 # p1 = Person()
33 # # 手动向对象添加name属性
34 # p1.name = '孙悟空'
35
36 # p2 = Person()
37 # p2.name = '猪八戒'
38
```

```
39 # p3 = Person()
40 # p3.name = '沙和尚'
41
42 # p3.say_hello()
43
44 p1 = Person('孙悟空')
45 p2 = Person('猪八戒')
46 p3 = Person('沙和尚')
47 p4 = Person('唐僧')
48 # p1.__init__() 不要这么做
49
50 # print(p1.name)
51 # print(p2.name)
52 # print(p3.name)
53 # print(p4.name)
54
55 p4.say_hello()
```

封装

```
1 # 封装是面向对象的三大特性之一
2 # 封装指的是隐藏对象中一些不希望被外部所访问到的属性或方法
3 # 如何隐藏一个对象中的属性？
4 #     - 将对象的属性名，修改为一个外部不知道的名字
5 # 如何获取（修改）对象中的属性？
6 #     - 需要提供一个getter和setter方法使外部可以访问到属性
7 #     - getter 获取对象中的指定属性（get_属性名）
8 #     - setter 用来设置对象的指定属性（set_属性名）
9 # 使用封装，确实增加了类的定义的复杂程度，但是它也确保了数据的安全性
10 # 1.隐藏了属性名，使调用者无法随意的修改对象中的属性
11 # 2.增加了getter和setter方法，很好的控制的属性是否是只读的
12 #     如果希望属性是只读的，则可以直接去掉setter方法
13 #     如果希望属性不能被外部访问，则可以直接去掉getter方法
14 # 3.使用setter方法设置属性，可以增加数据的验证，确保数据的值是正确的
15 # 4.使用getter方法获取属性，使用setter方法设置属性
16 #     可以在读取属性和修改属性的同时做一些其他的处理
17 # 5.使用getter方法可以表示一些计算的属性
```

```
18
```

```
19 class Dog:
20     '''
21     表示狗的类
22     '''
23     def __init__(self , name , age):
24         self.hidden_name = name
25         self.hidden_age = age
26
27     def say_hello(self):
28         print('大家好，我是 %s'%self.hidden_name)
29
30     def get_name(self):
31         '''
32         get_name()用来获取对象的名字属性
33         '''
34         # print('用户读取了属性')
35         return self.hidden_name
36
37     def set_name(self , name):
38         # print('用户修改了属性')
```



```
39         self.hidden_name = name
40
41     def get_age(self):
42         return self.hidden_age
43
44     def set_age(self , age):
45         if age > 0 :
46             self.hidden_age = age
47
48
49 d = Dog('旺财',8)
50
51 # d.say_hello()
52
53 # 调用setter来修改name属性
54 d.set_name('小黑')
55 d.set_age(-10)
56
57 # d.say_hello()
58 print(d.get_age())
```

```
1 class Rectangle:
2     '''
3     表示矩形的类
4     '''
5     def __init__(self,width,height):
6         self.hidden_width = width
7         self.hidden_height = height
8
9     def get_width(self):
10        return self.hidden_width
11
12    def get_height(self):
13        return self.hidden_height
14
15    def set_width(self , width):
16        self.hidden_width = width
17
18    def set_height(self , height):
19        self.hidden_height = height
20
21    def get_area(self):
22        return self.hidden_width * self.hidden_height
23
24 # r = Rectangle(5,2)
25 # r.set_width(10)
26 # r.set_height(20)
27
28 # print(r.get_area())
29
30
31 # 可以为对象的属性使用双下划线开头, __xxx
32 # 双下划线开头的属性, 是对象的隐藏属性, 隐藏属性只能在类的内部访问, 无法通过对象访问
33 # 其实隐藏属性只不过是Python自动为属性改了一个名字
34 # 实际上是将名字修改为了, _类名__属性名 比如 __name -> _Person__name
35 # class Person:
36 #     def __init__(self,name):
37 #         self.__name = name
38
```

```
39 #     def get_name(self):
40 #         return self.__name
41
42 #     def set_name(self , name):
43 #         self.__name = name
44
45 # p = Person('孙悟空')
46
47 # print(p.__name) __开头的属性是隐藏属性，无法通过对象访问
48 # p.__name = '猪八戒'
49 # print(p._Person__name)
50 # p._Person__name = '猪八戒'
51
52 # print(p.get_name())
53
54 # 使用__开头的属性，实际上依然可以在外部访问，所以这种方式我们一般不用
55 # 一般我们会将一些私有属性（不希望被外部访问的属性）以_开头
56 # 一般情况下，使用_开头的属性都是私有属性，没有特殊需要不要修改私有属性
57 class Person:
58     def __init__(self,name):
59         self._name = name
60
61     def get_name(self):
62         return self._name
63
64     def set_name(self , name):
65         self._name = name
66
67 p = Person('孙悟空')
68
69 print(p._name)
```

```
1 class Person:
2     def __init__(self,name,age):
3         self._name = name
4         self._age = age
5
6     # property装饰器，用来将一个get方法，转换为对象的属性
7     # 添加为property装饰器以后，我们就可以像调用属性一样使用get方法
8     # 使用property装饰的方法，必须和属性名是一样的
9     @property
10    def name(self):
11        print('get方法执行了~~~')
12        return self._name
13
14    # setter方法的装饰器: @属性名.setter
15    @name.setter
16    def name(self , name):
17        print('setter方法调用了')
18        self._name = name
19
20    @property
21    def age(self):
22        return self._age
23
24    @age.setter
25    def age(self , age):
26        self._age = age
27
28
29
30 p = Person('猪八戒',18)
31
32 p.name = '孙悟空'
33 p.age = 28
34
35 print(p.name,p.age)
36
```

继承

```
1 # 继承
2
3 # 定义一个类 Animal（动物）
4 # 这个类中需要两个方法：run() sleep()
5 class Animal:
6     def run(self):
7         print('动物会跑~~~')
8
9     def sleep(self):
10        print('动物睡觉~~~')
11
12    # def bark(self):
13    #     print('动物嚎叫~~~')
14
15 # 定义一个类 Dog（狗）
16 # 这个类中需要三个方法：run() sleep() bark()
17 # class Dog:
18 #     def run(self):
19 #         print('狗会跑~~~')
20
21 #     def sleep(self):
22 #         print('狗睡觉~~~')
23
24 #     def bark(self):
25 #         print('汪汪汪~~~')
26
27 # 有一个类，能够实现我们需要的大部分功能，但是不能实现全部功能
28 # 如何能让这个类来实现全部的功能呢？
29 # ① 直接修改这个类，在这个类中添加我们需要的功能
30 #     - 修改起来会比较麻烦，并且会违反OCP原则
31 # ② 直接创建一个新的类
32 #     - 创建一个新的类比较麻烦，并且需要大量的进行复制粘贴，会出现大量的重复性代码
33 # ③ 直接从Animal类中来继承它的属性和方法
34 #     - 继承是面向对象三大特性之一
35 #     - 通过继承我们可以使一个类获取到其他类中的属性和方法
36 #     - 在定义类时，可以在类名后的括号中指定当前类的父类（超类、基类、super）
37 #         子类（衍生类）可以直接继承父类中的所有的属性和方法
38 #
```

```
39 # 通过继承可以直接让子类获取到父类的方法或属性，避免编写重复性的代码，并且也符合OCP原则
40 # 所以我们经常需要通过继承来对一个类进行扩展
41
42 class Dog(Animal):
43     def bark(self):
44         print('汪汪汪~~~')
45
46     def run(self):
47         print('狗跑~~~~')
48
49 class Hashiqi(Dog):
50     def fan_sha(self):
51         print('我是一只傻傻的哈士奇')
52
53 d = Dog()
54 h = Hashiqi()
55
56 # d.run()
57 # d.sleep()
58 # d.bark()
59
60 # r = isinstance(d , Dog)
61 # r = isinstance(d , Animal)
62 # print(r)
63
64 # 在创建类时，如果省略了父类，则默认父类为object
65 # object是所有类的父类，所有类都继承自object
66 class Person(object):
67     pass
68
69 # issubclass() 检查一个类是否是另一个类的子类
70 # print(issubclass(Animal , Dog))
71 # print(issubclass(Animal , object))
72 # print(issubclass(Person , object))
73
74 # isinstance()用来检查一个对象是否是一个类的实例
75 # 如果这个类是这个对象的父类，也会返回True
76 # 所有的对象都是object的实例
77 print(isinstance(print , object))
```



```
1 class Animal:
2     def __init__(self, name):
3         self._name = name
4
5     def run(self):
6         print('动物会跑~~~')
7
8     def sleep(self):
9         print('动物睡觉~~~')
10
11     @property
12     def name(self):
13         return self._name
14
15     @name.setter
16     def name(self, name):
17         self._name = name
18
19 # 父类中的所有方法都会被子类继承，包括特殊方法，也可以重写特殊方法
20 class Dog(Animal):
21
22     def __init__(self, name, age):
23         # 希望可以调用父类的__init__来初始化父类中定义的属性
24         # super() 可以用来获取当前类的父类，
25         # 并且通过super()返回对象调用父类方法时，不需要传递self
26         super().__init__(name)
27         self._age = age
28
29     def bark(self):
30         print('汪汪汪~~~')
31
32     def run(self):
33         print('狗跑~~~~')
34
35     @property
36     def age(self):
37         return self._age
```

```
38
39     @age.setter
40     def age(self, age):
41         self._age = age
42
43 d = Dog('旺财', 18)
44
45 print(d.name)
46 print(d.age)
```

多重继承

```

1 class A(object):
2     def test(self):
3         print('AAA')
4
5 class B(object):
6     def test(self):
7         print('B中的test()方法~~')
8
9     def test2(self):
10        print('BBB')
11
12 # 在Python中是支持多重继承的，也就是我们可以为一个类同时指定多个父类
13 # 可以在类名的()后边添加多个类，来实现多重继承
14 # 多重继承，会使子类同时拥有多个父类，并且会获取到所有父类中的方法
15 # 在开发中没有特殊的情况，应该尽量避免使用多重继承，因为多重继承会让我们的代码过于复杂
16 # 如果多个父类中有同名的方法，则会现在第一个父类中寻找，然后找第二个，然后找第三个。。。
17 # 前边父类的方法会覆盖后边父类的方法
18 class C(A,B):
19     pass
20
21 # 类名.__bases__ 这个属性可以用来获取当前类的所有父类
22 # print(C.__bases__) (<class '__main__.B'>,)
23 # print(B.__bases__) (<class 'object'>,)
24
25 # print(C.__bases__) # (<class '__main__.A'>, <class '__main__.B'>)
26
27 c = C()
28
29 c.test()
30

```

重写

```
1 # 继承
2
3 # 定义一个类 Animal（动物）
4 # 这个类中需要两个方法：run() sleep()
5 class Animal:
6     def run(self):
7         print('动物会跑~~~')
8
9     def sleep(self):
10        print('动物睡觉~~~')
11
12
13 class Dog(Animal):
14     def bark(self):
15         print('汪汪汪~~~')
16
17     def run(self):
18         print('狗跑~~~~')
19
20
21 # 如果在子类中如果有和父类同名的方法，则通过子类实例去调用方法时，
22 # 会调用子类的方法而不是父类的方法，这个特点我们成为叫做方法的重写（覆盖，override）
23 # 创建Dog类的实例
24 # d = Dog()
25
26 # d.run()
27
28 # 当我们调用一个对象的方法时，
29 # 会优先去当前对象中寻找是否具有该方法，如果有则直接调用
30 # 如果没有，则去当前对象的父类中寻找，如果父类中有则直接调用父类中的方法，
31 # 如果没有，则去父类的父类中寻找，以此类推，直到找到object，如果依然没有找到，则报错
32 class A(object):
33     def test(self):
34         print('AAA')
35
36 class B(A):
37     def test(self):
```

```
38         print('BBB')
39
40     class C(B):
41         def test(self):
42             print('CCC')
43
44     # 创建一个c的实例
45     c = C()
46     c.test()
47
48
```

多态

```
1 # 多态是面向对象的三大特征之一
2 # 多态从字面上理解是多种形态
3 # 狗（狼狗、藏獒、哈士奇、古牧。。。）
4 # 一个对象可以以不同的形态去呈现
5
6 # 定义两个类
7 class A:
8     def __init__(self,name):
9         self._name = name
10
11     @property
12     def name(self):
13         return self._name
14
15     @name.setter
16     def name(self,name):
17         self._name = name
18
19 class B:
20     def __init__(self,name):
21         self._name = name
22
23     def __len__(self):
24         return 10
25
26     @property
27     def name(self):
28         return self._name
29
30     @name.setter
31     def name(self,name):
32         self._name = name
33
34 class C:
35     pass
36
37
38 a = A('孙悟空')
```

```
39 b = B('猪八戒')
40 c = C()
41
42 # 定义一个函数
43 # 对于say_hello()这个函数来说，只要对象中含有name属性，它就可以作为参数传递
44 # 这个函数并不会考虑对象的类型，只要有name属性即可
45 def say_hello(obj):
46     print('你好 %s'%obj.name)
47
48 # 在say_hello_2中我们做了一个类型检查，也就是只有obj是A类型的对象时，才可以正常使用，
49 # 其他类型的对象都无法使用该函数，这个函数就违反了多态
50 # 违反了多态的函数，只适用于一种类型的对象，无法处理其他类型对象，这样导致函数的适应性非常的差
51 # 注意，向isinstance()这种函数，在开发中一般是不会使用的！
52 def say_hello_2(obj):
53     # 做类型检查
54     if isinstance(obj , A):
55         print('你好 %s'%obj.name)
56 # say_hello(b)
57 # say_hello_2(b)
58
59 # 鸭子类型
60 # 如果一个东西，走路像鸭子，叫声像鸭子，那么它就是鸭子
61
62 # len()
63 # 之所以一个对象能通过len()来获取长度，是因为对象中具有一个特殊方法__len__
64 # 换句话说，只要对象中具有__len__特殊方法，就可以通过len()来获取它的长度
65 l = [1,2,3]
66 s = 'hello'
67
68 # print(len(l))
69 # print(len(s))
70 print(len(b))
71 print(len(c))
72
73 # 面向对象的三大特征：
74 # 封装
75 #     - 确保对象中的数据安全
76 # 继承
77 #     - 保证了对象的可扩展性
```

78 # 多态

79 # - 保证了程序的灵活性

类中的属性和方法


```
1 # 定义一个类
2 class A(object):
3
4     # 类属性
5     # 实例属性
6     # 类方法
7     # 实例方法
8     # 静态方法
9
10    # 类属性，直接在类中定义的属性是类属性
11    # 类属性可以通过类或类的实例访问到
12    # 但是类属性只能通过类对象来修改，无法通过实例对象修改
13    count = 0
14
15    def __init__(self):
16        # 实例属性，通过实例对象添加的属性属于实例属性
17        # 实例属性只能通过实例对象来访问和修改，类对象无法访问修改
18        self.name = '孙悟空'
19
20    # 实例方法
21    # 在类中定义，以self为第一个参数的方法都是实例方法
22    # 实例方法在调用时，Python会将调用对象作为self传入
23    # 实例方法可以通过实例和类去调用
24    # 当通过实例调用时，会自动将当前调用对象作为self传入
25    # 当通过类调用时，不会自动传递self，此时我们必须手动传递self
26    def test(self):
27        print('这是test方法~~~ ', self)
28
29    # 类方法
30    # 在类内部使用 @classmethod 来修饰的方法属于类方法
31    # 类方法的第一个参数是cls，也会被自动传递，cls就是当前的类对象
32    # 类方法和实例方法的区别，实例方法的第一个参数是self，而类方法的第一个参数是cls
33    # 类方法可以通过类去调用，也可以通过实例调用，没有区别
34    @classmethod
35    def test_2(cls):
36        print('这是test_2方法，他是一个类方法~~~ ', cls)
37        print(cls.count)
38
```

```
39     # 静态方法
40     # 在类中使用 @staticmethod 来修饰的方法属于静态方法
41     # 静态方法不需要指定任何的默认参数，静态方法可以通过类和实例去调用
42     # 静态方法，基本上是一个和当前类无关的方法，它只是一个保存到当前类中的函数
43     # 静态方法一般都是一些工具方法，和当前类无关
44     @staticmethod
45     def test_3():
46         print('test_3执行了~~~')
47
48
49     a = A()
50     # 实例属性，通过实例对象添加的属性属于实例属性
51     # a.count = 10
52     # A.count = 100
53     # print('A ',A.count)
54     # print('a ',a.count)
55     # print('A ',A.name)
56     # print('a ',a.name)
57
58     # a.test() 等价于 A.test(a)
59
60     # A.test_2() 等价于 a.test_2()
61
62     A.test_3()
63     a.test_3()
64
65
66
```

垃圾回收

```
1 # 就像我们生活中会产生垃圾一样，程序在运行过程当中也会产生垃圾
2 # 程序运行过程中产生的垃圾会影响到程序的运行的运行性能，所以这些垃圾必须被及时清理
3 # 没用的东西就是垃圾
4 # 在程序中没有被引用的对象就是垃圾，这种垃圾对象过多以后会影响到程序的运行的性能
5 # 所以必须进行及时的垃圾回收，所谓的垃圾回收就是讲垃圾对象从内存中删除
6 # 在Python中有自动的垃圾回收机制，它会自动将这些没有被引用的对象删除，
7 # 所以我们不用手动处理垃圾回收
8
9 class A:
10     def __init__(self):
11         self.name = 'A类'
12
13     # del是一个特殊方法，它会在对象被垃圾回收前调用
14     def __del__(self):
15         print('A()对象被删除了~~~',self)
16
17 a = A()
18 b = a # 又使用一个变量b，来引用a对应的对象
19
20 print(a.name)
21
22 # a = None # 将a设置为了None，此时没有任何的变量对A()对象进行引用，它就是变成了垃圾
23 # b = None
24 # del a
25 # del b
26 input('回车键退出...')
27
```

特殊方法

```
1 # 特殊方法，也称为魔术方法
2 # 特殊方法都是使用__开头和结尾的
3 # 特殊方法一般不需要我们手动调用，需要在一些特殊情况下自动执行
4
5 # 定义一个Person类
6 class Person(object):
7     """人类"""
8     def __init__(self, name , age):
9         self.name = name
10        self.age = age
11
12    # __str__ () 这个特殊方法会在尝试将对象转换为字符串的时候调用
13    # 它的作用可以用来指定对象转换为字符串的结果 （print函数）
14    def __str__(self):
15        return 'Person [name=%s , age=%d]'%(self.name,self.age)
16
17    # __repr__() 这个特殊方法会在对当前对象使用repr()函数时调用
18    # 它的作用是指定对象在 ‘交互模式’ 中直接输出的效果
19    def __repr__(self):
20        return 'Hello'
21
22    # object.__add__(self, other)
23    # object.__sub__(self, other)
24    # object.__mul__(self, other)
25    # object.__matmul__(self, other)
26    # object.__truediv__(self, other)
27    # object.__floordiv__(self, other)
28    # object.__mod__(self, other)
29    # object.__divmod__(self, other)
30    # object.__pow__(self, other[, modulo])
31    # object.__lshift__(self, other)
32    # object.__rshift__(self, other)
33    # object.__and__(self, other)
34    # object.__xor__(self, other)
35    # object.__or__(self, other)
36
37    # object.__lt__(self, other) 小于 <
38    # object.__le__(self, other) 小于等于 <=
```

```
39     # object.__eq__(self, other) 等于 ==
40     # object.__ne__(self, other) 不等于 !=
41     # object.__gt__(self, other) 大于 >
42     # object.__ge__(self, other) 大于等于 >=
43
44     # __len__()获取对象的长度
45
46     # object.__bool__(self)
47     # 可以通过bool来指定对象转换为布尔值的情况
48     def __bool__(self):
49         return self.age > 17
50
51     # __gt__会在对象做大于比较的时候调用，该方法的返回值将会作为比较的结果
52     # 他需要两个参数，一个self表示当前对象，other表示和当前对象比较的对象
53     # self > other
54     def __gt__(self, other):
55         return self.age > other.age
56
57
58 # 创建两个Person类的实例
59 p1 = Person('孙悟空',18)
60 p2 = Person('猪八戒',28)
61
62 # 打印p1
63 # 当我们打印一个对象时，实际上打印的是对象的中特殊方法 __str__()的返回值
64 # print(p1) # <__main__.Person object at 0x04E95090>
65 # print(p1)
66 # print(p2)
67
68 # print(repr(p1))
69
70 # t = 1,2,3
71 # print(t) # (1, 2, 3)
72
73 # print(p1 > p2)
74 # print(p2 > p1)
75
76 # print(bool(p1))
77
```

```
78 # if p1 :
79 #     print(p1.name,'已经成年了')
80 # else :
81 #     print(p1.name,'还未成年了')
82
```

模块

```
1 # 模块（module）
2 # 模块化，模块化指将一个完整的程序分解为一个一个小的模块
3 # 通过将模块组合，来搭建出一个完整的程序
4 # 不采用模块化，统一将所有的代码编写到一个文件中
5 # 采用模块化，将程序分别编写到多个文件中
6 # 模块化的有点：
7 #     ① 方便开发
8 #     ② 方便维护
9 #     ③ 模块可以复用！
10
11 # 在Python中一个py文件就是一个模块，要想创建模块，实际上就是创建一个python文件
12 # 注意：模块名要符号标识符的规范
13
14 # 在一个模块中引入外部模块
15 # ① import 模块名 （模块名，就是python文件的名称，注意不要py）
16 # ② import 模块名 as 模块别名
17 # - 可以引入同一个模块多次，但是模块的实例只会创建一个
18 # - import可以在程序的任意位置调用，但是一般情况下，import语句都会统一写在程序的开头
19 # - 在每一个模块内部都有一个__name__属性，通过这个属性可以获取到模块的名字
20 # - __name__属性值为 __main__ 的模块是主模块，一个程序中只会有一个主模块
21 #     主模块就是我们直接通过 python 执行的模块
22 import test_module as test
23
24 # print(test.__name__)
25 print(__name__)
```

```
1 # import m
2
3 # # 访问模块中的变量: 模块名.变量名
4 # # print(m.a , m.b)
5
6 # # m.test2()
7
8 # p = m.Person()
9
10 # print(p.name)
11
12 def test2():
13     print('这是主模块中的test2')
14
15
16 # 也可以只引入模块中的部分内容
17 # 语法 from 模块名 import 变量,变量....
18 # from m import Person
19 # from m import test
20 # from m import Person,test
21 # from m import * # 引入到模块中所有内容,一般不会使用
22 # p1 = Person()
23 # print(p1)
24 # test()
25 # test2()
26
27 # 也可以为引入的变量使用别名
28 # 语法: from 模块名 import 变量 as 别名
29 # from m import test2 as new_test2
30
31 # test2()
32 # new_test2()
33
34 from m import *
35 # print(_c)
36
37 # import xxx
38 # import xxx as yyy
```

```
39 # from xxx import yyy , zzz , fff
40 # from xxx import *
41 # from xxx import yyy as zz
42
```

包

```
1 # 包 Package
2 # 包也是一个模块
3 # 当我们模块中代码过多时，或者一个模块需要被分解为多个模块时，这时就需要使用到包
4 # 普通的模块就是一个py文件，而包是一个文件夹
5 # 包中必须要一个一个 __init__.py 这个文件，这个文件中可以包含有包中的主要内容
6 from hello import a , b
7
8 print(a.c)
9 print(b.d)
10
11 # __pycache__ 是模块的缓存文件
12 # py代码在执行前，需要被解析器先转换为机器码，然后再执行
13 # 所以我们在使用模块（包）时，也需要将模块的代码先转换为机器码然后再交由计算机执行
14 # 而为了提高程序运行的性能，python会在编译过一次以后，将代码保存到一个缓存文件中
15 # 这样在下次加载这个模块（包）时，就可以不再重新编译而是直接加载缓存中编译好的代码即可
16
```

异常

简介


```
1 # print('hello')
2 # try:
3 #     # try中放置的是有可能出现错误的代码
4 #     print(10/0)
5 # except:yi
6 #     # except中放置的是出错以后的处理防暑
7 #     print('哈哈哈，出错了~~~')
8 # else:
9 #     print('程序正常执行没有错误')
10 # print('你好')
11
12
13 # print(10/0)
14
15
16 def fn():
17     print('Hello fn')
18     print(a)
19     print(10/0)
20
21 def fn2():
22     print('Hello fn2')
23     fn()
24
25 def fn3():
26     print('Hello fn3')
27     fn2()
28
29 fn3()
30
```

异常对象

```
1 print('异常出现前')
2 l = []
3 try:
4     # print(c)
5     # l[10]
6     # 1 + 'hello'
7     print(10/0)
8 except NameError:
9     # 如果except后不跟任何的内容，则此时它会捕获到所有的异常
10    # 如果在except后跟着一个异常的类型，那么此时它只会捕获该类型的异常
11    print('出现 NameError 异常')
12 except ZeroDivisionError:
13    print('出现 ZeroDivisionError 异常')
14 except IndexError:
15    print('出现 IndexError 异常')
16 # Exception 是所有异常类的父类，所以如果except后跟的是Exception，他也会捕获到所有的异常
17 # 可以在异常类后边跟着一个 as xx 此时xx就是异常对象
18 except Exception as e :
19    print('未知异常',e,type(e))
20 finally :
21    print('无论是否出现异常，该子句都会执行')
22
23 print('异常出现后')
```

抛出异常

```
1 # 也可以自定义异常类，只需要创建一个类继承Exception即可
2 class MyError(Exception):
3     pass
4
5 def add(a,b):
6     # 如果a和b中有负数，就向调用处抛出异常
7     if a < 0 or b < 0:
8         # raise用于向外部抛出异常，后边可以跟一个异常类，或异常类的实例
9         # raise Exception
10        # 抛出异常的目的，告诉调用者这里调用时出现问题，希望你自己处理一下
11        # raise Exception('两个参数中不能有负数! ')
12        raise MyError('自定义的异常')
13
14    # 也可以通过if else来代替异常的处理
15    # return None
16    r = a + b
17    return r
18
19 print(add(-123,456))
```

文件

打开

```
1 # open(file, mode='r', buffering=-1, encoding=None, errors=None, newline=None,
   # closefd=True, opener=None)
2 # 使用open函数来打开一个文件
3 # 参数:
4 #   file 要打开的文件的名字（路径）
5 # 返回值:
6 #   返回一个对象，这个对象就代表了当前打开的文件
7
8 # 创建一个变量，来保存文件的名字
9 # 如果目标文件和当前文件在同一级目录下，则直接使用文件名即可
10 file_name = 'demo.txt'
11
12 # 在windows系统使用路径时，可以使用/来代替 \
13 # 或者可以使用 \\ 来代替 \
14 # 或者也可以使用原始字符串
15 file_name = 'hello\\demo.txt'
16 file_name = r'hello\demo.txt'
17
18 # 表示路径，可以使用..来返回一级目录
19 file_name = '../hello/demo.txt'
20
21 # 如果目标文件距离当前文件比较远，此时可以使用绝对路径
22 # 绝对路径应该从磁盘的根目录开始书写
23 file_name = r'C:\Users\lilichao\Desktop\hello.txt'
24 file_obj = open(file_name) # 打开 file_name 对应的文件
25 print(file_obj)
26
```

关闭

```
1 # 打开文件
2 file_name = 'demo.txt'
3
4 # 调用open()来打开文件
5 file_obj = open(file_name)
6 file_obj = open(file,encoding="utf-8")
7
8 # 当我们获取了文件对象以后，所有的对文件的操作都应该通过对象来进行
9 # 读取文件中的内容
10 # read()方法，用来读取文件中的内容，它会将内容全部保存为一个字符串返回
11 content = file_obj.read()
12 print(content)
13
14 # 关闭文件
15 # 调用close()方法来关闭文件
16 file_obj.close()
17
18 # with ... as 语句
19 with open(file_name) as file_obj :
20 # 在with语句中可以直接使用file_obj来做文件操作
21 # 此时这个文件只能在with中使用，一旦with结束则文件会自动close()
22 print(file_obj.read())
23
24
25 file_name = 'hello'
26
27 try:
28     with open(file_name) as file_obj :
29         print(file_obj.read())
30 except FileNotFoundError:
31     print(f'{file_name} 文件不存在~~')
```

读取

```

1 file_name = 'demo2.txt'
2
3 try:
4     # 调用open()来打开一个文件，可以将文件分成两种类型
5     # 一种，是纯文本文件（使用utf-8等编码编写的文本文件）
6     # 一种，是二进制文件（图片、mp3、ppt等这些文件）
7     # open()打开文件时，默认是以文本文件的形式打开的，但是open()默认的编码为None
8     # 所以处理文本文件时，必须要指定文件的编码
9     with open(file_name,encoding='utf-8') as file_obj:
10         # 通过 read() 来读取文件中的内容
11         # 如果直接调用read()它会将文本文件的所有内容全部都读取出来
12         # 如果要读取的文件较大的话，会一次性将文件的内容加载到内存中，容易导致内存泄漏
13         # 所以对于较大的文件，不要直接调用read()
14         # help(file_obj.read)
15         # read()可以接收一个size作为参数，该参数用来指定要读取的字符的数量
16         # 默认值为-1，它会读取文件中的所有字符
17         # 可以为size指定一个值，这样read()会读取指定数量的字符，
18         #     每一次读取都是从上一次读取到位置开始读取的
19         #     如果字符的数量小于size，则会读取剩余所有的
20         #     如果已经读取到了文件的最后了，则会返回''空串
21         # content = file_obj.read(-1)
22         content = file_obj.read(6)
23         content = file_obj.read(6)
24         content = file_obj.read(6)
25         content = file_obj.read(6)
26         # print(content)
27         # print(len(content))
28 except FileNotFoundError :
29     print(f'{file_name} 这个文件不存在! ')
30
31 # 读取大文件的方式
32 file_name = 'demo.txt'
33
34 try:
35     with open(file_name,encoding='utf-8') as file_obj:
36         # 定义一个变量，来保存文件的内容
37         file_content = ''
38         # 定义一个变量，来指定每次读取的大小

```

```
39     chunk = 100
40     # 创建一个循环来读取文件内容
41     while True:
42         # 读取chunk大小的内容
43         content = file_obj.read(chunk)
44
45         # 检查是否读取到了内容
46         if not content:
47             # 内容读取完毕，退出循环
48             break
49
50         # 输出内容
51         # print(content,end='')
52         file_content += content
53
54 except FileNotFoundError :
55     print(f'{file_name} 这个文件不存在! ')
56
57
58 print(file_content)
```

```
1 import pprint
2 import os
3 file_name = 'demo.txt'
4
5 with open(file_name , encoding='utf-8') as file_obj:
6     # readline()
7     # 该方法可以用来读取一行内容
8     # print(file_obj.readline(),end='')
9     # print(file_obj.readline())
10    # print(file_obj.readline())
11
12    # readlines()
13    # 该方法用于一行一行的读取内容，它会一次性将读取到的内容封装到一个列表中返回
14    # r = file_obj.readlines()
15    # pprint.pprint(r[0])
16    # pprint.pprint(r[1])
17    # pprint.pprint(r[2])
18
19    for t in file_obj:
20        print(t)
21
22
```



```
1 # with open('demo.txt','rb') as file_obj:
2 #     # print(file_obj.read(100))
3 #     # print(file_obj.read(30))
4
5 #     # seek() 可以修改当前读取的位置
6 #     file_obj.seek(55)
7 #     file_obj.seek(80,0)
8 #     file_obj.seek(70,1)
9 #     file_obj.seek(-10,2)
10 #     # seek()需要两个参数
11 #     #     第一个 是要切换到的位置
12 #     #     第二个 计算位置方式
13 #     #     可选值:
14 #     #         0 从头计算, 默认值
15 #     #         1 从当前位置计算
16 #     #         2 从最后位置开始计算
17
18 #     print(file_obj.read())
19
20 #     # tell() 方法用来查看当前读取的位置
21 #     print('当前读取到了 -->',file_obj.tell())
22
23 with open('demo2.txt','rt' , encoding='utf-8') as file_obj:
24     # print(file_obj.read(100))
25     # print(file_obj.read(30))
26
27     # seek() 可以修改当前读取的位置
28     file_obj.seek(9)
29     # seek()需要两个参数
30     #     第一个 是要切换到的位置
31     #     第二个 计算位置方式
32     #     可选值:
33     #         0 从头计算, 默认值
34     #         1 从当前位置计算
35     #         2 从最后位置开始计算
36
37     print(file_obj.read())
38
```

```
39 # tell() 方法用来查看当前读取的位置
40 print('当前读取到了 -->',file_obj.tell())
```

写入

```
1 file_name = 'demo5.txt'
2
3 # 使用open()打开文件时必须指定打开文件所要做的操作（读、写、追加）
4 # 如果不指定操作类型，则默认是 读取文件 ， 而读取文件时是不能向文件中写入的
5 # r 表示只读的
6 # w 表示是可写的，使用w来写入文件时，如果文件不存在会创建文件，如果文件存在则会截断文件
7 # 截断文件指删除原来文件中的所有内容
8 # a 表示追加内容，如果文件不存在会创建文件，如果文件存在则会向文件中追加内容
9 # x 用来新建文件，如果文件不存在则创建，存在则报错
10 # + 为操作符增加功能
11 # r+ 即可读又可写，文件不存在会报错
12 # w+
13 # a+
14 # with open(file_name , 'w' , encoding='utf-8') as file_obj:
15 # with open(file_name , 'r+' , encoding='utf-8') as file_obj:
16 with open(file_name , 'x' , encoding='utf-8') as file_obj:
17     # write()来向文件中写入内容，
18     # 如果操作的是一个文本文件的话，则write()需要传递一个字符串作为参数
19     # 该方法会可以分多次向文件中写入内容
20     # 写入完成以后，该方法会返回写入的字符的个数
21     file_obj.write('aaa\n')
22     file_obj.write('bbb\n')
23     file_obj.write('ccc\n')
24     r = file_obj.write(str(123)+'123123\n')
25     r = file_obj.write('今天天气真不错')
26     print(r)
27
```

```
1 file_name = 'c:/Users/lilichao/Desktop/告白气球.flac'
2
3 # 读取模式
4 # t 读取文本文件（默认值）
5 # b 读取二进制文件
6
7 with open(file_name , 'rb') as file_obj:
8     # 读取文本文件时，size是以字符为单位的
9     # 读取二进制文件时，size是以字节为单位
10    # print(file_obj.read(100))
11
12    # 将读取到的内容写出来
13    # 定义一个新的文件
14    new_name = 'aa.flac'
15
16    with open(new_name , 'wb') as new_obj:
17
18        # 定义每次读取的大小
19        chunk = 1024 * 100
20
21        while True :
22            # 从已有的对象中读取数据
23            content = file_obj.read(chunk)
24
25            # 内容读取完毕，终止循环
26            if not content :
27                break
28
29            # 将读取到的数据写入到新对象中
30            new_obj.write(content)
```

使用with语句和csv模块读取CSV文件

```
1 import csv
2
3 with open('data.csv', mode='r') as file:
4     csvFile = csv.reader(file)
5     for row in csvFile:
6         print(row)
```

删除文件

```
1 os.remove('aa.txt')
```

重命名

```
1 # os.rename('旧名字', '新名字') 可以对一个文件进行重命名，也可以用来移动一个文件
2 os.rename('aa.txt', 'bb.txt')
3 os.rename('bb.txt', 'c:/users/lilichao/desktop/bb.txt')
```

其他操作

环境变量的读取和设置

```
1 import os
2
3 # 读取环境变量
4 print("PATH:", os.environ["PATH"])
5 # 设置环境变量
6 os.environ["NEW_VAR"] = "NewValue"
7 print("NEW_VAR:", os.environ["NEW_VAR"])
```

路径操作

```
1 import os
2
3 path = "/path/to/some/file.txt"
4 dirname = os.path.dirname(path)
5 basename = os.path.basename(path)
6 print("Directory:", dirname)
7 print("Basename:", basename)
```

环境变量的读取和设置

```
1 import os
2
3 # 读取环境变量
4 print("PATH:", os.environ["PATH"])
5 # 设置环境变量
6 os.environ["NEW_VAR"] = "NewValue"
7 print("NEW_VAR:", os.environ["NEW_VAR"])
```

目录操作

```
1 import os
2 from pprint import pprint
3
4 # 创建目录
5 os.mkdir("aaa") # 在当前目录下创建一个名字为 aaa 的目录
6
7 # 删除目录
8 os.rmdir('abc')
9
10 # os.listdir() 获取指定目录的目录结构
11 # 需要一个路径作为参数，会获取到该路径下的目录结构，默认路径为 . 当前目录
12 # 该方法会返回一个列表，目录中的每一个文件（夹）的名字都是列表中的一个元素
13 r = os.listdir()
14
15 # os.getcwd() 获取当前所在的目录
16 r = os.getcwd()
17
18 # os.chdir() 切换当前所在的目录 作用相当于 cd
19 os.chdir('c:/')
20
21 r = os.getcwd()
```

JSON字符串与字典的转换

字符串转换为字典

```
1 方法1(转换字符串)
2 json_str="xxxxx" #字符串
3 dict=json.loads(json_str)
4 方法二(从文件读取)
5 res=json.load(open('file.txt','r',encoding='utf-8')))
```

字典转换为字符串

```
1 #方法一(获取字符串)
2 dict={}
3 json_str=json.dumps(dict,ensure_ascii=True,indent=4)
4
5 #方法二(直接写入文件)
6 json.dump(dict,open('file.txt','w',encoding='utf-8'),ensure_ascii=True,indent=4)
```

案例

```
1 import json
2
3 if __name__ == '__main__':
4     dict={"k1":"v1","k2":"v2"}
5     str=json.dumps(dict,indent=4,ensure_ascii=True)
6     print(str)
7     json.dump(dict,open('jsonfile.txt','w',encoding="utf-8"),indent=4,ensure_ascii=True)
```

```
1 import json
2
3 if __name__ == '__main__':
4     dict=json.load(open('jsonfile.txt','r',encoding='utf-8'))
5     print(type(dict))
6     print(dict)
7
8     with open('jsonfile.txt','r',encoding="utf-8") as f :
9         str=f.read()
10        print(str)
11        dict=json.loads(str)
12        print(type(dict))
```

jsonpath

安装

```
1 pip install jsonpath
```

引入

```
1 import jsonpath
```

基本语法

案例

```
1 {  
2     "k1": "v1",  
3     "k2": "v2",  
4     "k3": {  
5         "k33": "v33",  
6         "k34": {  
7             "k44": "v44"  
8         }  
9     }  
10 }
```



```

1 import json
2 from jsonpath import jsonpath
3 if __name__ == '__main__':
4     with open('jsonfile.txt','r',encoding="utf-8") as f :
5         jsonstr=f.read()
6         print(jsonstr)
7     dict=json.load(open('jsonfile.txt','r',encoding='utf-8'))
8     print(type(dict))
9     print(dict)
10    k1=jsonpath(dict,'$.k1')
11    #k44=jsonpath(dict,'$..k44')
12    k44=jsonpath(dict,'$.k3.k34.k44')
13    print("k1=", k1, "k44=", k44)

```

正则表达式

正则表达式查找字符串

```

1 import re
2
3 pattern = r'\b[A-Za-z][A-Za-z0-9_]*\b'
4 text = "Hello, this is a test string with username: JohnDoe"
5 matches = re.findall(pattern, text)
6 print(matches)

```

Jupyter

安装

```
1 pip install jupyter -i http://mirrors.aliyun.com/pypi/simple/ --trusted-host
  mirrors.aliyun.com
2
3 ##异常修复
4 python -m pip install --upgrade --force pip -i http://mirrors.aliyun.com/pypi/simple/
  --trusted-host mirrors.aliyun.com
5 pip install setuptools==33.1.1 -i http://mirrors.aliyun.com/pypi/simple/ --trusted-
  host mirrors.aliyun.com
6
7 ##代码提示
8 pip install jupyter_contrib_nbextensions
9 jupyter contrib nbextensions install --user --skip-running-check
```

启动

```
1 #默认端口
2 Jupyter notebook
3
4 #指定端口
5 jupyter notebook --port 9999 [--ip="*"] [--no-browser]
```

快捷键

```
1 Ctrl+Enter 运行
2 Shift+Enter 运行本单元,选中下一单元
3 Alt+Enter 运行本单元,其后插入一单元
```

其他技巧

Python和Sublime的整合

```
1 1.在Sublime中执行Python代码，ctrl + b 自动在Sublime内置的控制台中执行
2     这种执行方式，在某些版本的Sublime中对中文支持不好，并且不能使用input()函数
3
4 2.使用SublimeREPL来运行python代码
5     安装完成，设置快捷键，希望按f5则自动执行当前的Python代码
6     { "keys": ["f5"], "caption": "SublimeREPL:Python", "command":
  "run_existing_window_command", "args": {"id": "repl_python_run", "file":
  "config/Python/Main.sublime-menu"}},
```

Sublime sftp 之 Host key verification failed

```
1 打开注册表
2 Ctrl + R -> regedit
3 删除相关
4 Key HKEY_CURRENT_USER\Software\SimonTatham\PuTTY\SshHostKeys
```

print不换行

```
1 print(" ",end='')
```

input

```
1 # input()函数
2 # 该函数用来获取用户的输入
3 # input()调用后，程序会立即暂停，等待用户输入
4 # 用户输入完内容以后，点击回车程序才会继续向下执行
5 # 用户输入完成以后，其所输入的的内容会以返回值得形式返回
6 # 注意：input()的返回值是一个字符串
7 # input()函数中可以设置一个字符串作为参数，这个字符串将会作为提示文字显示
8 # a = input('请输入任意内容：')
9 # print('用户输入的内容是:',a)
10 # input()也可以用于暂时阻止程序结束
11
12 # 获取用户输入的用户名
13 username = input('请输入你的用户名:')
14 # 判断用户名是否是admin
15 if username == 'admin' :
16     print('欢迎管理员光临！')
```

time模块

```
1 from time import *
2 begin = time()
```

文档字符串

```
1 # help()是Python中的内置函数
2 # 通过help()函数可以查询python中的函数的用法
3 # 语法: help(函数对象)
4 # help(print) # 获取print()函数的使用说明
5
6 # 文档字符串 (doc str)
7 # 在定义函数时, 可以在函数内部编写文档字符串, 文档字符串就是函数的说明
8 # 当我们编写了文档字符串时, 就可以通过help()函数来查看函数的说明
9 # 文档字符串非常简单, 其实直接在函数的第一行写一个字符串就是文档字符串
10 def fn(a:int,b:bool,c:str='hello') -> int:
11     '''
12     这是一个文档字符串的示例
13
14     函数的作用: 。。。。。
15     函数的参数:
16         a, 作用, 类型, 默认值。。。。
17         b, 作用, 类型, 默认值。。。。
18         c, 作用, 类型, 默认值。。。。
19     '''
20     return 10
21 help(fn)
```

标准程序库

```
1 # 开箱即用
2 # 为了实现开箱即用的思想，Python中为我们提供了一个模块的标准库
3 # 在这个标准库中，有很多很强大的模块我们可以直接使用，
4 # 并且标准库会随Python的安装一同安装
5 # sys模块，它里面提供了一些变量和函数，使我们可以获取到Python解析器的信息
6 # 或者通过函数来操作Python解析器
7 # 引入sys模块
8 import sys
9
10 # pprint 模块它给我们提供了一个方法 pprint() 该方法可以用来对打印的数据做简单的格式化
11 import pprint
12
13 # sys.argv
14 # 获取执行代码时，命令行中所包含的参数
15 # 该属性是一个列表，列表中保存了当前命令的所有参数
16 # print(sys.argv)
17
18 # sys.modules
19 # 获取当前程序中引入的所有模块
20 # modules是一个字典，字典的key是模块的名字，字典的value是模块对象
21 # pprint.pprint(sys.modules)
22
23 # sys.path
24 # 他是一个列表，列表中保存的是模块的搜索路径
25 # ['C:\\Users\\lilichao\\Desktop\\resource\\course\\lesson_06\\code',
26 # 'C:\\dev\\python\\python36\\python36.zip',
27 # 'C:\\dev\\python\\python36\\DLLs',
28 # 'C:\\dev\\python\\python36\\lib',
29 # 'C:\\dev\\python\\python36',
30 # 'C:\\dev\\python\\python36\\lib\\site-packages']
31 # pprint.pprint(sys.path)
32
33 # sys.platform
34 # 表示当前Python运行的平台
35 # print(sys.platform)
36
37 # sys.exit()
38 # 函数用来退出程序
```

```
39 # sys.exit('程序出现异常，结束！')
40 # print('hello')
41
42 # os 模块让我们可以对操作系统进行访问
43 import os
44
45 # os.environ
46 # 通过这个属性可以获取到系统的环境变量
47 # pprint.pprint(os.environ['path'])
48
49 # os.system()
50 # 可以用来执行操作系统的名字
51 # os.system('dir')
52 os.system('notepad')
```

打包为EXE

安装 pyinstaller

```
1 pip install pyinstaller
```

执行打包

```
1 #main.py 为程序入口
2 pyinstaller -F main.py
```

打包相关选项

```
1 #单文件打包
2 pyinstaller.exe -F file.py
3
4 #多文件打包
5 pyinstaller.exe -F file.py
6
7 #不显示控制台窗口
8 pyinstaller.exe -F file.py --noconsole
9
10 #指定生成文件名称
11 pyinstaller.exe -F file.py -n myfile.exe
12
13 #指定生成文件图标
14 pyinstaller.exe -F file.py -i file.icon
```

视频合成

安装MoviePy

```
1 pip install MoviePy
```

案例

```
1 from moviepy.editor import ffmpeg_tools
2 if __name__ == '__main__':
3     video01="D:/BaiduNetdiskDownload/江恩理论/nb20220423_江恩理论1.mp4"
4     video02="D:/BaiduNetdiskDownload/江恩理论/nb20220423_江恩理论2.mp4"
5     target_video="target.mp4"
6     ffmpeg_tools.ffmpeg_merge_video_audio(video01,video02,target_video)
```


特定用法

参数解析

```

1  import sys
2
3  from modules.SSHCmdExecutor import SSHCmdExecutor
4  import argparse
5
6
7  if __name__ == '__main__':
8
9      parser = argparse.ArgumentParser(prog=sys.argv[0])
10     parser.add_argument("--host", help="IP", required=True, type=str)
11     parser.add_argument("--port", help="端口", required=False, default=22)
12     parser.add_argument("--user", help="用户名", required=True, type=str)
13     parser.add_argument("--password", help="密码", required=True, type=str)
14     parser.add_argument("--command", help="执行命令", required=True, type=str)
15     _args=parser.parse_args()
16     print(sys.argv)
17     print("IP="+_args.host)
18     exec=SSHCmdExecutor(ip=_args.host,
19                          port=_args.port,
20                          username=_args.user,
21                          password=_args.password,
22                          command=_args.command)
23     exec.doExec()
24
25     -----
26
27 import argparse
28 import sys
29 if __name__ == '__main__':
30     parser = argparse.ArgumentParser(prog=sys.argv[0])
31     #可以指定选项别名    parser.add_argument("--host", "-H",
32     help="IP", required=False, type=str, default="hostIP")
33     parser.add_argument("--port", "-P", help="端
34     口", required=False, default=22, dest='hostPort')
35     parser.add_argument("--user", "-U", help="用户名", required=False, type=str)
36
37     #自动开关选项    parser.add_argument("--ok", "-y", help="是否
38     OK", action='store_true', dest='isOK')

```

```
36     parser.add_argument("--no", "-n", help="是否不Ok", action='store_false',
37     dest="isNo")
38
39     #指定数值类选项    parser.add_argument("--num", help="数量", type=int,
40     dest="number", default=80)
41
42     #添加互斥参数    group=parser.add_mutually_exclusive_group()
43
44     group.add_argument("-d", "--down", help="down", dest="isDown", action='store_true')
45     group.add_argument("-t", "--top", help="top", dest="isTop", action='store_true')
46
47     # 添加枚举选项    parser.add_argument("-c", "--choice", help="选
48     择", dest="choice", choices=[1,2,3], type=int)
49
50     params=parser.parse_args()
51     print(params)
```

子命令解析

```

1 import argparse
2 import sys
3
4 if __name__ == '__main__':
5     parser=argparse.ArgumentParser(prog=sys.argv[0],description="参数解析测试程序")
6     parser.add_argument('--foo',action='store_true',help='foo help')
7     #通过parse_name区分调用的是那个子命令
8     subparasers=parser.add_subparsers(help='b command comment',dest="paser_name")
9     paraser_a=subparasers.add_parser('cmda',help='a command comment')
10    paraser_a.add_argument('--bar',dest='bar',help='bar
value',default='0',required=True)
11    paraser_a.add_argument('--add',dest='addvalue',help='add
value',default='0',required=False)
12    paraser_a.add_argument('--dec',dest='decvalue',help='dec
value',default='0',required=False)
13
14
15    paraser_b = subparasers.add_parser('cmdb', help='b command comment')
16    paraser_b.add_argument('--bar', dest='bar', help='bar value', default='0',
required=True)
17    paraser_b.add_argument('--addb', dest='addvalueb', help='add value', default='0',
required=False)
18    paraser_b.add_argument('--decb', dest='decvalueb', help='dec value', default='0',
required=False)
19
20
21    args=parser.parse_args()
22
23
24    if args.paser_name=='cmda' :
25        print("cmd a process ")
26    elif args.paser_name=='cmdb' :
27        print("cmd b process")
28
29    print(args)

```

SSH执行命令

```
1 import paramiko
2
3 import paramiko
4 class SSHCmdExecutor:
5     def __init__(self, ip, port, username, password, command):
6         self.ip=ip
7         self.port=port
8         self.username=username
9         self.password=password
10        self.command=command
11
12    def doExec(self):
13        try:
14            # 建立一个sshclient对象
15            ssh = paramiko.SSHClient()
16            # 允许将信任的主机自动加入到host_allow 列表，此方法必须放在connect方法的前面
17
18            ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
19            # 调用connect方法连接服务器
20            ssh.connect(hostname=self.ip, port=self.port, username=self.username,
21                password=self.password)
22            # 执行命令
23            stdin, stdout, stderr = ssh.exec_command(self.command)
24            # 结果放到stdout中，如果有错误将放到stderr中
25            print(stdout.read().decode())
26            # 关闭连接
27            ssh.close()
28        except Exception as e:
29            print(e)
```

SSH文件传输

```

1  import paramiko
2
3  class SSHFileExecutor:
4      def __init__(self, ip, port, username, password, remotefile, localfile, type):
5          self.ip = ip
6          self.port = port
7          self.username = username
8          self.password = password
9          self.remotefile = remotefile
10         self.localfile = localfile
11         self.type=type
12
13     def doExec(self):
14         try:
15             # 建立一个sshclient对象
16             client = paramiko.SSHClient()
17             # 允许将信任的主机自动加入到host_allow 列表，此方法必须放在connect方法的前面
18
19             client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
20             # 调用connect方法连接服务器
21             client.connect(hostname=self.ip, port=self.port, username=self.username,
22                             password=self.password)
23             scp=client.open_sftp();
24             if(self.type=="get" or self.type=="GET"):
25                 scp.get(self.remotefile, self.localfile)
26             elif(self.type=="put" or self.type=="PUT"):
27                 scp.put(self.localfile, self.remotefile)
28             else:
29                 print("错误的操作类型")
30             scp.close()
31             client.close()
32         except Exception as e:
33             print(e)

```

PDF转word

模块安装

```
1 pip instal pdf2docx
```

源码实现

```
1 from pdf2docx import Converter
2 if __name__ == '__main__':
3     pdf_file=r'd:/tmp/wxh.pdf'    word_file=r'd:/tmp/wxh.docx'    cv =
    Converter(pdf_file)
4     cv.convert(word_file,start=0,end=None)
5     cv.close()
```

执行系统命令

fopen

```
1 import os
2 print(os.popen("cat /etc/passwd").read())
```

itertools库的用法

```
1 sma1=range(20,61,4)
2 sma2=range(180,281,4)
3
4 for SMA1,SMA2 in product(sma1,sma2):
5     print("SMA1=",SMA1," SMA2=",SMA2)
```

爬虫

安装模块

```
1 pip install requests
2 or
3 conda install requests
```

导入模块

```
1 import requests
```

基本使用方法

```
1 url = "https://www.baidu.com"
2 response = requests.get(url)
3 print(response.status_code) #获取状态码
4 print(response.text)
5 print(response.content) #content为原始的字节数据
6 print(response.content.decode()) #默认使用utf-8进行解码
7 #print(response.content.decode('GBK')) #默认使用GBK进行解码
8
9 #response.headers 响应头
10 print(response.headers)
11
12 #response.cookies 获取Cookie
13 print(response.cookies)
14
15 #response.request.headers 获取请求头
16 print(response.request.headers)
```

字符串编码与解码


```
1     str = "你好 Python"
2     btr = str.encode() #编码
3     print(btr)
4     print(type(btr))
5     str2 = btr.decode() #解码
6     print(str2)
7     print(type(str2))
```

自动探测编码

安装模块

```
1 pip install chardet
```

案例

```
1 import chardet
2 import requests
3 r=requests.get("https://www.sanhetouzi.com/")
4 encoding=chardet.detect(r.content)
5 print(type(encoding['encoding']))
6 print(encoding['encoding'])
```

带请求头的请求

```
1 headers = {
2     "User-Agent" : "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
    (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36"
3 }
4 url = "https://www.baidu.com"
5 response = requests.get(url,headers=headers)
6 print(response.content.decode())
```

图片下载

```
1 headers = {
2     "User-Agent" : "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
    (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36"
3 }
4 url = "https://www.baidu.com/img/PCtm_d9c8750bed0b3c7d089fa7d55720d6cf.png"
5 response = requests.get(url,headers=headers)
6 with open("baidu.jpg","wb") as f:
7     f.write(response.content)
```

网页下载

```
1 headers = {
2     "User-Agent" : "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
    (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36"
3 }
4 url = "https://www.baidu.com"
5 response = requests.get(url,headers=headers)
6 with open("baidu.html","w",encoding="utf-8") as f:
7     f.write(response.content.decode())
```

pyquery

安装

```
1 pip install -y pyquery
```

解析html

```
1 from pyquery import PyQuery as pq
2 html = '''<div>      <ul>          <li>Apple</li>          <li>Banana</li>
   <li>Orange</li>      </ul></div>'''
3 doc=pq(html)
4 selector="li:eq(1)" #selector="li:eq(1)"
5 r=doc(selector)
6 rt=r.text()
7 print(rt)
```

xpath

选取节点

谓语句（条件过滤）

选取未知节点

模糊匹配

获取数据

获取文本数据

获取属性值

安装lxml

```
1 pip install lxml
```

python 通过xpath解析数据案例

```
1     headers = {
2         "User-Agent" : "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36"
3     }
4     url = "https://movie.douban.com/top250"
5     response=requests.get(url,headers=headers)
6     page=response.content.decode()
7     # print(page)
8     html=etree.HTML(page)
9     lis=html.xpath("//ol/li")
10    for li in lis:
11        title=li.xpath('.//div[@class="hd"]/a/span[1]/text()')
12        desc=li.xpath('.//div[@class="bd"]/p/text()')
13        score=li.xpath('.//div[@class="bd"]//span[@class="rating_num"]/text()')
14        print("title=",title[0],"desc=",(desc[0]+"").strip(),"score=",score)
```

带参数的GET请求

```
1     headers = {
2         "User-Agent" : "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36"
3     }
4     url = "https://movie.douban.com/top250"
5     params={
6         "start" : "25",
7         "filter": ""
8     }
9     response=requests.get(url,headers=headers,params=params)
10    page=response.content.decode()
11    print(page)
```

post请求

表单参数的传递方法

```
1 request.post(url,data="字典参数")
```

json参数的传递方法

```
1 request.post(url,json="字典参数")
```

解析json数据

```
1     headers = {
2         "User-Agent" : "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36"
3     }
4     url="https://www.amtb.cn/v2/medias/01/4/1/236734"
5     response=requests.get(url,headers=headers)
6     json=response.json()
7     mp3_list=json['jiangtanginfo']['mp3list'];
8     for mp3 in mp3_list:
9         print(mp3["id"],'--',mp3["itemid"],'- ',mp3['mediapath'])
```

模拟登录

使用cookie+session鉴权的网站模拟登录的流程

实现流程

- 1、传递账号密码，进行登录
- 2、登录之后保存cookie(返回时在响应头的set-cookie字段中)
- 3、请求其他的页面时，携带cookie

案例

案例1

```

1  headers = {
2      "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36" ,
3      "Referer": "https://passport.china.com/logon"
4  }
5  login_url = "https://passport.china.com/logon"
6  params = {
7      "username" : "17775990925",
8      "passport" : "a546245426"
9  }
10 # 模拟登录
11 response=requests.post(url=login_url,data=params,headers=headers)
12 #登录成功以后请求其他页面
13 requests.get("url2",headers=headers,cookies=response.cookies)
14

```

案例2

```

1  headers = {
2      "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36" ,
3      "Referer": "https://passport.china.com/logon"
4  }
5  login_url = "https://passport.china.com/logon"
6  params = {
7      "username" : "17775990925",
8      "passport" : "a546245426"
9  }
10 # 模拟登录
11 response=requests.post(url=login_url,data=params,headers=headers)
12 #登录成功以后请求其他页面
13 # requests.get("url2",headers=headers,cookies=response.cookies)
14 http=requests.session()
15 resp2=http.get(url="https://passport.china.com/",headers=headers)
16 print(resp2.content.decode())

```

使用token鉴权的网站模拟登录的流程

实现流程

- 1 1、传递账号密码，进行登录
- 2 2、登录之后保存token(返回的时候，在响应体中)
- 3 3、请求其他的页面时，携带token

python-docx

安装

```
1 pip install python-docx
```

引入

```
1 from docx import Document
```

文档操作

新建文档

```
1 from docx import Document
2 docx=Document()
3 docx.save("file.docx")
```


打开文档

```
1 #直接打开
2 docx=Document("file.docx")
3
4 #通过文件句柄打开
5 with open("file.docx","rb") as f :
6     docx=Document(f)
```

doc转docx

安装pywin32

```
1 pip install pywin32
```

实现

```

1 from win32com import client
2 import os
3
4 def doc_to_docx(path):
5     print("路径: {}".format(path))
6     if not os.path.isabs(path):
7         print(f"不是绝对路径")
8         return
9     if not os.path.exists(path):
10        print("文件不存在")
11        return
12    if os.path.splitext(path)[1] != ".doc":
13        print("文件类型不对")
14        return
15
16    app=client.Dispatch("Word.Application")
17    app.DisplayAlerts=False
18    document=app.Documents.Open(path)
19    document.SaveAs(os.path.splitext(path)[0]+".docx",12)
20    document.Close()
21    app.Quit()

```

设置纸张的大小与方向

```

1 docx=Document("file.docx")
2 for section in docx.sections:
3     section.page_width=Cm(21)
4     section.page_height=Cm(29.7)
5     print(section.page_width.cm)
6     print(section.page_height.cm)
7     #横向
8     section.orientation=WD_ORIENTATION.LANDSCAPE
9     #纵向
10    section.orientation=WD_ORIENTATION.PORTRAIT

```

设置页边距与装订线

```
1 docx=Document("file.docx")
2 for section in docx.sections:
3     section.top_margin=Cm(0.5)
4     section.bottom_margin=Cm(0.5)
5     section.left_margin=Cm(0.5)
6     section.right_margin=Cm(0.5)
7     section.gutter=Cm(0.0)
```

段落操作

```
1 #添加段落
2 doc=Document()
3 p1=doc.add_paragraph("Content")
4
5 #文档段落数
6 print(len(doc.paragraphs))
7
8 #修改段落
9 p1=doc.add_paragraph("Content")
10 p1.text="Modifi content"
11
12 #删除段落
13 p1.text=""
14 p1.clear()
15
16 #彻底删除段落
17 def deleteParagraph(paragraph):
18     p=paragraph._element
19     p.getparent().remove(p)
20     p._p=p._element=None
21
22 #段落前插入内容
23 p1.insert_paragraph_before("Insert Content")
24
25 #插入分页符
26 p1=doc.add_paragraph("段落内容")
27 doc.add_page_break()
```

段落的样式

段落对齐

```
1 from docx import Document
2 from docx.enum.text import WD_PARAGRAPH_ALIGNMENT
3
4 if __name__ == '__main__':
5     docx = Document()
6     p1 = docx.add_paragraph("左对齐")
7     p2 = docx.add_paragraph("右对齐")
8     p3 = docx.add_paragraph("居中对齐")
9     p4 = docx.add_paragraph("两端对齐")
10    p5 = docx.add_paragraph("分散对齐")
11
12    p1.alignment = WD_PARAGRAPH_ALIGNMENT.LEFT
13    p2.alignment = WD_PARAGRAPH_ALIGNMENT.RIGHT
14    p3.alignment = WD_PARAGRAPH_ALIGNMENT.CENTER
15    p4.alignment = WD_PARAGRAPH_ALIGNMENT.JUSTIFY
16    p5.alignment = WD_PARAGRAPH_ALIGNMENT.DISTRIBUTE
17
18    docx.save("./dir/file.docx")
```

查看支持所有段落样式名称

```
1 from docx import Document
2 from docx.enum.style import WD_STYLE_TYPE
3
4 def printParagraphStyleName():
5     docx = Document()
6     styles=docx.styles
7     paragraph_styles=[ s for s in styles if s.type == WD_STYLE_TYPE.PARAGRAPH ]
8     for style in paragraph_styles:
9         print(style.name)
```

设置段落样式

```
1 from docx import Document
2 from docx.enum.style import WD_STYLE_TYPE
3
4 if __name__ == '__main__':
5     docx = Document()
6     p1=docx.add_paragraph("标题1")
7     p1.style="Heading 1"
8     p2 = docx.add_paragraph("序号列表1")
9     p2.style="List Number"
10    p2 = docx.add_paragraph("序号列表2")
11    p2.style = "List Number"
12
13    p3 = docx.add_paragraph("无序列表1")
14    p3.style="List Bullet"
15    p3 = docx.add_paragraph("无序列表2")
16    p3.style = "List Bullet"
17
18    #删除样式
19
20    p3.style.delete()
```

并发编程

创建多线程的方法

案例

案例1

```
1 import threading
2
3 def my_func(a,b):
4     print(f"Hello World:{a}+{b}={a+b}")
5
6
7 if __name__ == '__main__':
8     t=threading.Thread(target=my_func,args=(10,10))
9     t.start()
10    t.join()
```

生产者与消费者

Queue

```
1 #导入
2 import queue
3
4 #创建Queue
5 q=queue.Queue()
6
7 #添加
8 q.put(item)
9
10 #获取
11 item=q.get()
12
13 #查询状态
14 q.qsize() #元素多少
15 q.empty() #判断是否为空
16 q.full()  #判断是否为满
```

案例

```

1  import queue
2  import requests
3  from bs4 import BeautifulSoup
4  import time
5  import random
6  import threading
7
8
9  urls=[
10      f"https://www.cnblogs.com/#p{page}"
11      for page in range(1,50+1)
12  ]
13
14  def craw(url):
15      r=requests.get(url)
16      return r.text
17
18  def parase(html):
19      soup = BeautifulSoup(html, "html.parser")
20      links = soup.find_all("a",class_="post-item-title")
21      return [ (link["href"],link.get_text()) for link in links ]
22
23
24  def do_craw(url_queue:queue.Queue,html_queue:queue.Queue):
25      while True:
26          url = url_queue.get()
27          html = craw(url)
28          html_queue.put(html)
29          print(threading.current_thread().name,f'craw
{url}','url_queue.size=",url_queue.qsize())
30          time.sleep(random.randint(1,2))
31
32
33  def do_parase(html_queue:queue.Queue,fout):
34      while True:
35          html=html_queue.get()
36          results = parase(html)
37          for result in results:
38              fout.write(str(result)+"\n")

```



```
39         print(threading.current_thread().name,
40               f'result.size',
41               len(result), "html_queue.size=", html_queue.qsize())
42         time.sleep(random.randint(1,2))
43
44
45 if __name__ == '__main__':
46     url_queue=queue.Queue()
47     html_queue=queue.Queue()
48     for url in urls:
49         url_queue.put(url)
50
51     for idx in range(3):
52         t=threading.Thread(target=do_craw,args=
53 (url_queue,html_queue),name=f"craw{idx}")
54         t.start()
55
56     fout=open("02.data.txt","w")
57     for idx in range(2):
58         t=threading.Thread(target=do_parase,args=(html_queue,fout),name=f"parse{idx}")
59         t.start()
60
```

Lock解决线程安全问题

try-finally模式

with模式

案例

```
1 import threading
2 import time
3 lock = threading.Lock()
4
5 class Account:
6     def __init__(self,balance):
7         self.balance=balance
8
9 def draw(account:Account,amount):
10     with lock:
11         if account.balance > amount:
12             time.sleep(0.1)
13             print(threading.current_thread().name,"取钱成功")
14             account.balance -= amount
15             print(threading.current_thread().name, "余额",account.balance)
16         else:
17             print(threading.current_thread().name, "取钱失败余额不足")
18
19 if __name__ == '__main__':
20     account=Account(1000)
21     ta = threading.Thread(name="ta",target=draw,args=(account,800))
22     tb = threading.Thread(name="tb", target=draw, args=(account, 800))
23     ta.start()
24     tb.start()
```

线程池

原理

ThreadPoolExecutor的使用语法

案例1

```

1 import concurrent.futures
2 import requests
3 from bs4 import BeautifulSoup
4
5 header={
6     "Origin":"https://www.cnblogs.com",
7     "Referer":"https://www.cnblogs.com/",
8     "User-Agent":"Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
    like Gecko) Chrome/124.0.0.0 Safari/537.36"
9 }
10
11
12 urls=[
13     f"https://www.cnblogs.com/#p{page}"
14     for page in range(1,50+1)
15 ]
16
17 def crawl(url):
18     r=requests.get(url,headers=header)
19     return r.text
20
21 def parse(html):
22     soup = BeautifulSoup(html, "html.parser")
23     links = soup.find_all("a",class_="post-item-title")
24     return [ (link["href"],link.get_text()) for link in links ]
25
26
27 if __name__ == '__main__':
28     with concurrent.futures.ThreadPoolExecutor() as pool :
29         htmls=pool.map(crawl,urls)
30         htmls=list(zip(urls,htmls))
31         for url,html in htmls :
32             print(url,len(html))
33
34     print("crawl over")
35
36     with concurrent.futures.ThreadPoolExecutor() as pool :
37         futures={}
38         for url,html in htmls :

```

```
39         future=pool.submit(parase,html)
40         futures[future]=url
41     for future,url in futures.items() :
42         print(url,future.result())
43     print("=====")
44     for future in concurrent.futures.as_completed(futures) :
45         url = futures[future]
46         print(url,future.r esult())
47
48
```

案例2

```
1 import json
2 import time
3 import flask
4 from concurrent.futures import ThreadPoolExecutor
5
6 pool = ThreadPoolExecutor()
7 app = flask.Flask(__name__)
8
9
10 def read_api():
11     time.sleep(0.3)
12     return "api result"
13
14
15 def read_file():
16     time.sleep(0.1)
17     return "file result"
18
19 def read_db():
20     time.sleep(0.2)
21     return "db result"
22
23 @app.route("/")
24 def index():
25     result_file=pool.submit(read_file)
26     result_db=pool.submit(read_db)
27     result_api=pool.submit(read_api)
28
29     return json.dumps({
30         "result_file": result_file.result(),
31         "result_db" : result_db.result(),
32         "result_api": result_api.result()
33     })
34
35 if __name__ == '__main__':
36     app.run()
```

multiprocessing

案例1

```
1 import math
2 from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor
3 import time
4
5 PRIMES=[112272535095293] * 100
6
7 def is_prime(n):
8     if n < 2:
9         return False
10    if n == 2:
11        return True
12    if n % 2 == 0:
13        return False
14    sqrt_n = int(math.floor(math.sqrt(n)))
15    for i in range(3, sqrt_n + 1, 2):
16        if n % i == 0:
17            return False
18    return True
19
20
21 def single_thread():
22     for number in PRIMES :
23         is_prime(number)
24
25 def multi_thread():
26     with ThreadPoolExecutor() as pool:
27         pool.map(is_prime,PRIMES)
28
29 def multi_process():
30     with ProcessPoolExecutor() as pool:
31         pool.map(is_prime,PRIMES)
32
33 if __name__ == '__main__':
34     start=time.time()
35     single_thread()
36     end = time.time()
37     print("single thread,cost:",end-start)
38
```

```
39     start=time.time()
40     multi_thread()
41     end = time.time()
42     print("multi thread,cost:",end-start)
43
44     start=time.time()
45     multi_process()
46     end = time.time()
47     print("multi process,cost:",end-start)
```

异步IO

`pip install asyncio,aiohttp`

案例


```

1 import asyncio
2 import aiohttp
3 import time
4
5 urls=[
6     f"https://www.cnblogs.com/#p{page}"
7     for page in range(1,50+1)
8 ]
9
10
11 async def async_craw(url):
12     print("craw url :",url)
13     async with aiohttp.ClientSession() as session :
14         async with session.get(url) as resp :
15             result = await resp.text()
16             print(f"craw url : {url},{len(result)}")
17
18
19 if __name__ == '__main__':
20     start=time.time()
21     loop=asyncio.get_event_loop()
22     tasks = [ loop.create_task(async_craw(url)) for url in urls ]
23     loop.run_until_complete(asyncio.wait(tasks))
24     end=time.time()
25     print("use time seconds:",end-start)

```

信号量

信号量（英语：Semaphore）又称为信号量、旗语

是一个同步对象，用于保持在0至指定最大值之间的一个计数值。

- 当线程完成一次对该semaphore对象的等待（wait）时，该计数值减一；
- 当线程完成一次对semaphore对象的释放（release）时，计数值加一。
- 当计数值为0，则线程等待该semaphore对象不再能成功直至该semaphore对象变成signaled状态
- semaphore对象的计数值大于0，为signaled状态；计数值等于0，为nonsignaled状态。

使用方式1:

```
sem = asyncio.Semaphore(10)

# ... later
async with sem:
    # work with shared resource
```

使用方式2:

```
sem = asyncio.Semaphore(10)

# ... later
await sem.acquire()
try:
    # work with shared resource
finally:
    sem.release()
```

Selenium

<https://selenium-python.readthedocs.io/index.html>

安装

```
1 pip install selenium
```

浏览器驱动下载

```
1 https://chromedriver.storage.googleapis.com/index.html
```

基本使用

```
1 from selenium import webdriver
2 if __name__ == '__main__':
3     #启动浏览器
4     driver=webdriver.Chrome()
5     url="https://www.baidu.com"
6     #访问页面
7     driver.get(url)
8     #获取页面源码
9     print(driver.page_source)
10    #页面最大化
11    driver.maximize_window()
12    #刷新页面
13    driver.refresh()
14    #关闭浏览器和driver程序
15
16    #保存源码
17    with open("baidu.html","w",encoding='utf-8') as f :
18        f.write(driver.page_source)
19
20    # 截图
21    driver.save_screenshot("baidu.png")
22    driver.quit()
```

Driver相关属性

元素定位

```
1 #元素名称
2 driver.find_element(By.NAME,"name")
3
4 # 元素ID
5 driver.find_element(By.ID,"id")
6
7 # 标签名称
8 driver.find_element(By.TAG_NAME,"tagname")
9
10 # 类名
11 driver.find_element(By.CLASS_NAME,"classname")
12
13 # xpath
14 driver.find_element(By.XPATH,"xpath")
```

元素的属性与操作

元素属性

元素操作

获取程序运行路径

```
1 import os
2 import sys
3
4
5 if __name__ == '__main__':
6     BASE_DIR1=os.path.dirname(os.path.abspath(__file__))
7     #由于打包问题，有限选用该方法
8     BASE_DIR2=os.path.dirname(os.path.realpath(sys.argv[0]))
9     print(f"BASE_DIR1 {BASE_DIR1} \r\nBASE_DIR2 {BASE_DIR2}")
```

路径拼接

```
1 os.path.join(BASE_DIR,"account.txt")
```

笔记

20231201

```
1 1 序列长度    len()
2 2 ASCII转字符 chr()
3 3 字符转编码  ord()
4 4 打印函数    print(value=...,sep=' ',end="\n",file=None)
5 5 文件操作    fp=open("file.txt","w")
6                fp.close()
7 6 输入 x=input("提示文字:")
8 7 字符串转换为整数 int()
9 8 注释
10 8.1 单行注释 #
11 8.2 多行注释 """ """ or ''' '''
12 8.3 中文声明注释 #coding=编码 (#coding=utf-8)
13 9 数值转换为字符串 str()
14 10 获取python中的保留字
15 import keyword
16 print(keyword.kwlist)
17
18 11 标识符：标识符由字符（中，英文），下划线，数字组成，不能以数字开头，不能为保留字，大小写敏感。
```

程序的流程图描述方式

20231203

```
1
2 1 链式赋值
3 a=b=c=d=100
4 2 字符串分解赋值
5 a,b,c,d='room'
6
7 3 eval类型转换与计算
8 3.1类型转换
9 n=eval(input('请输入数字'))
10 3.2计算
11 a=1
12 b=2
13 c=eval("a+b")
14 print("c=",c)
15
16
17 4 数值转换
18 4.1 数值转Bool 0->False 其他-True
19 if __name__ == '__main__':
20     #数值与Bool类型转换
21     n=98
22     if n%2:
23         print(n,'是奇数')
24     if not n%2:
25         print(n,'是偶数')
26
27 4.2 字符串转Bool 空->False 非空->True
28 if __name__ == '__main__':
29     x=input("输入一个字符串: ")
30     if x:
31         print("x是一个非空字符串")
32
33 5 if语句块中只有一句代码,可将该代码写在冒号后面
34 a=10
35 b=5
36 if a>b: max=a
37 print(a,"和",b,"的最大值为",max)
38
```

```
39 6 if双分支结构简化
40 if __name__ == '__main__':
41     num=eval(input("输入号码: "))
42     r="中奖了" if num==100 else "未中奖"
43     print(r)
```

44

45 7 多分支if

46

```
47 if __name__ == '__main__':
48     score=eval(input("输入成绩: "))
49     if score<0 or score>100:
50         print("成绩有误")
51     elif 0<=score<=60:
52         print("E")
53     elif 60<score<=70:
54         print("D")
55     elif 70<score<=80:
56         print("C")
57     elif 80<score<=90:
58         print("B")
59     else:
60         print("A")
```

61

62 8 多个条件连接 and or not

63

64 9 Python3.11的新特性: 模式匹配

65

```
66 if __name__ == '__main__':
67     score=input("输入成绩: ")
68     match score:
69         case 'A':
70             print("优秀")
71         case 'B':
72             print("良好")
73         case 'C':
74             print("中等")
75         case 'D':
76             print("及格")
77         case 'E':
```

```
78         print("不及格")
79
80 10 循环结构
81 for 循环变量 in 遍历对象:
82     语句块
83
84 for 循环变量 in 遍历对象:
85     语句块
86 else:
87     正常循环结束后执行
88
89
90 while 表达式:
91     语句块
92
93 while 表达式:
94     语句块
95 else
96     正常循环结束后执行
97
98
99 if __name__ == '__main__':
100     #遍历字符串
101     for i in "Hello":
102         print("i=",i)
103
104     #遍历整数序列
105     #range(n,m) 产生一个整数序列, 包含n,不包含m (n<m)
106     for i in range(1,11):
107         print("i=",i)
108
109     #寻找水仙花数
110     for i in range(100,999):
111         ge=i%10
112         sh=i//10%10
113         ba=i//100
114         if i == ge**3+sh**3+ba**3:
115             print("i=",i)
116
```



```
117     sum=0
118     for i in range(1,11):
119         sum+=i
120     else:
121         print("for 正常结束,没有遇到break终止循环")
122         print("sum=",sum)
```

```
123
124     start = 1
125     total = 0
126     end = 100
127     while start <= end:
128         total += start
129         start+=1
130     else:
131         print("total=", total)
```

132

133

134 **11 pass** 空语句, 起占位的作用